

SOFTWARE DEVELOPMENT PROJECT

Final Report

Campus Maintenance Request & Work-Order Management System

Prepared by:

Student Name

Kamogelo Bantsheng

Student Number

202311455

Department of Computer Science & Information Technology

Sol Plaatje University

Submitted to:

Dr Silas Verkijika

Lecturer: Project (NPRJ740)

Department of Computer Science & Information Technology

Sol Plaatje University

24-26 Scanlan St, New Park, Kimberley, 8301

6 November 2026

DOCUMENT CONTROL

Version 5.0, 4 November 2026. If this report disagrees with an earlier phase document, this report is the one that counts.

Item	Value
Document	NPRJ740 Final Project Report
System name	Campus Maintenance Request & Work-Order Management System
Author and student number	Kamogelo Bantsheng, 202311455
Sections 6 to 9	Written up from the final build. Results in Sections 7 and 9 come from real test runs; the files are in docs/evidence/.

The five roles are Resident, Technician, Supervisor, Warden (residence management) and Admin (system administrator). A request is the fault report, and it carries the only lifecycle status. A work order is the assignment of a request to a Technician and has no status of its own. I use these words the same way everywhere.

These are the changes since the Phase 1 proposal and my earlier drafts, with the reason for each.

Ref	Change	Reason
CH-01	Requester renamed Resident. Warden added as a fifth, read-only role.	The users in the case study are residents. Management wants to monitor the work but should not decide who does it.
CH-02	Scope narrowed to a Moroka Hall pilot.	All my evidence comes from one residence. The data model keeps a Residence table so more can be added later.
CH-03	JWT replaced by server-side sessions (ADR-03).	A deactivated user or a changed role must stop working at once.
CH-04	Unique constraint alone replaced by a row lock, expected-state check and partial unique index (ADR-02).	A unique constraint does not stop two conflicting updates, such as two reassignments.
CH-05	Work orders no longer have their own status.	Two status fields can disagree, and I would have had to test every combination.
CH-06	“Immutable” audit log is now “append-only”.	A database superuser can still change the table, so “immutable” was a claim I could not defend (LIM-03).
CH-07	Draft rule BR-05 “Emergency Fallback” removed and replaced by a fixed notice on the report form.	It had no requirement, use case or screen behind it.
CH-08	Cancel, safety flag, master-data screens, notifications and attachments moved to Deferred or Excluded (FR-19 to FR-23).	None is needed for a success criterion, and I would rather finish fewer things properly.

Ref	Change	Reason
CH-09	Success criteria cut to five. Identifiers fixed and cross-referenced (Section 4.5).	So every requirement can be followed from problem evidence to test.
CH-10	A Warden may report a fault for a Resident, and answer questions on that report.	A Resident who cannot reach their account can still get a fault reported. Every other Warden write is refused.
CH-11	A Resident reports only against their assigned room and common areas.	A student should not report faults in another student's room.
CH-12	Two-factor sign-in was built, then removed. It stays Excluded (Section 3.3).	It was outside the agreed scope. Audit rows written while it existed are kept, because the log is append-only.

CONTENTS

DOCUMENT CONTROL 2

EXECUTIVE SUMMARY 5

1. PROJECT CONTEXT 6

2. PROBLEM, PURPOSE AND SUCCESS CRITERIA 7

2.1 Problem Statement 7

2.2 Project Purpose 7

2.3 Success Criteria 8

3. SOLUTION OVERVIEW AND SCOPE 8

3.1 Solution Overview 8

3.2 Final Release Scope 10

3.3 Excluded and Deferred Capabilities 10

3.4 Known Limitations and Open Issues 11

4. SYSTEM REQUIREMENTS SPECIFICATION 12

4.1 Functional Requirements 12

4.2 Non-Functional Requirements 15

4.3 Business Rules 16

4.4 Acceptance Criteria for Included Requirements 17

4.5 Requirements Traceability Matrix 20

5. SYSTEM DESIGN SPECIFICATION 21

5.1 Use Cases and Workflows 21

5.2 System Architecture and Technology Stack 27

5.3 Key Architecture and Design Decisions 29

5.4 Application Class Design 31

5.5 User Interface and Interaction Design 33

5.6 Data Design 46

5.7 Security, Privacy and Access Control 52

5.8 Deployment and Runtime Design 54
6. SYSTEM IMPLEMENTATION 55
6.1 Codebase and Repository Structure 55
6.2 Implementation of Core Capabilities 57
6.3 APIs, Integrations and External Dependencies 58
6.4 AI / Machine-Learning Component 59
7. VERIFICATION AND TESTING 59
7.1 Test Approach 59
7.2 Functional and Business-Rule Verification 59
7.3 Non-Functional and Security Verification 63
7.4 Defects and Known Issues 64
8. SYSTEM SETUP AND OPERATION 65
9. SOLUTION EVALUATION 67
10. REFERENCES 67
11. APPENDICES 68
11.1 GitHub Repository 68
11.2 Deployed System 68
11.3 Additional Supporting Material 68

EXECUTIVE SUMMARY

At Moroka Hall Residence at Sol Plaatje University, students report broken taps, lights and locks through WhatsApp groups or by telling someone. A report can pass through several people before a technician hears about it. Students cannot see what happened to their report, technicians have no proper job list, and management cannot tell which faults keep coming back or how long repairs take.

This project replaces that process with a web application. A resident reports a fault. A supervisor decides what happens to it and gives it to a technician. The technician records the repair, and the supervisor checks it before closing it. Wardens, who manage the residence,

follow progress on a read-only dashboard, and an administrator looks after the user accounts. Every step is saved, and only the right person can take it.

The system is built with React, Django and PostgreSQL and runs in Docker containers. Most of my design effort went into making the workflow reliable. Two supervisors cannot hand out the same job, a job cannot be marked complete without notes, a failed save never leaves half-finished data, and the history cannot be changed through the application. Notifications, single sign-on, cost tracking and a mobile app were left out on purpose (Section 3.3).

Overall outcome: every Included feature (FR-01 to FR-18) was delivered. On the final build 124 backend tests, 10 client tests and 6 browser journeys pass, including all 450 combinations of request state, action and role. With 5,000 requests and 30 simulated users, 95% of API calls finished in 130 ms against a 1 second target. All five success criteria were achieved (Section 9). A code audit before submission found 15 significant defects, listed with their fixes in Section 7.4.

1. PROJECT CONTEXT

Domain. Facility maintenance in university student housing.

Basis. The project is based on a real place, Moroka Hall Residence at Sol Plaatje University in Kimberley. It is an academic prototype: the residence does not use it, and I tested it with seeded data. The requirements come from two interviews held on 31 August 2026, with a facilities maintenance worker and a fourth-year resident. No residence or facilities staff reviewed the requirements or a prototype.

Five roles use the system.

Role	Who	What they do in the system
Resident	A student living at the residence	Reports faults, follows their own requests and answers questions from the supervisor.
Supervisor	Maintenance supervisor in the facilities team	Triages requests, assigns and reassigns Technicians, and verifies finished work. The only role that decides what work is done and by whom.
Technician	Maintenance worker	Does the assigned work and records completion notes. Sees only their own jobs.
Warden	Residence management	Watches their own residence read-only, and can report a fault for a Resident who cannot reach their account. Cannot influence who gets which job.
Admin	System administrator	Creates and manages user accounts and reads the audit log. Has no say in maintenance decisions.

2. PROBLEM, PURPOSE AND SUCCESS CRITERIA

2.1 Problem Statement

At Moroka Hall, maintenance requests travel by WhatsApp group or word of mouth. In the Phase 1 interviews the route to a technician sometimes had up to five hands in it, and none of these channels keeps a record you can search. Residents are affected because their faults stay unrepaired. Technicians are affected because they have no ordered job list. The facilities and residence managers are affected because they cannot see the backlog or the faults that repeat.

A chat thread cannot give what is missing here: one record per fault, a fixed lifecycle and a named person responsible. That is why software is justified.

Table 2.1: Problem evidence. Each finding has an ID that is carried into the requirements and the traceability matrix (Section 4.5).

ID	Finding	Evidence source	Consequence	Addressed by
P-1	Requests reach a technician through up to five intermediaries.	Semi-structured interviews, in person: maintenance worker and resident, 31 Aug 2026	Requests are delayed or lost and nobody is accountable.	FR-05, FR-08 to FR-10; SC-1
P-2	Residents cannot see the status of a report.	Semi-structured interview, in person: resident, 31 Aug 2026	Repeat reports and follow-up messages; loss of trust.	FR-05, FR-06; SC-1
P-3	Technicians have no formal, prioritised job list.	Semi-structured interview, in person: maintenance worker, 31 Aug 2026	Work is unordered and unfinished work is invisible.	FR-10 to FR-15; SC-1, SC-3
P-4	Management has no history of faults or repair times.	Semi-structured interviews, in person: maintenance worker and resident, 31 Aug 2026	Recurring faults and slow repairs go unnoticed.	FR-16 to FR-18; SC-4

The interview questions and the summary of answers are in the Phase 1 document, Section 9.

2.2 Project Purpose

The project builds a web system that keeps one record for every maintenance fault, from the first report to a verified repair. Each report is stored and has exactly one current state. Once assigned, it has exactly one responsible Technician. People see only what their role allows, and a history of who did what and when is kept.

2.3 Success Criteria

I judged the project against five outcomes. Each one can be checked, and Section 9 reports on them using the results from Section 7.

ID	Success criterion	How it is checked
SC-1	**Core workflow.** A fault can go from report to verified closure, including the clarification, rejection, reassignment and reopen paths, and every step outside the lifecycle in Table 5.1 is refused.	State × action × role test (TC-35) and end-to-end test (TC-42).
SC-2	**Access control.** Each role can do only what it is allowed to. No Resident can see another Resident's request and Wardens cannot change anything, apart from reporting a fault for a Resident.	Authorisation matrix test (NT-01): zero unauthorised successes.
SC-3	**Concurrency and data integrity.** Conflicting simultaneous actions on one request give exactly one success, a request never has two active Technicians, and multi-step changes are all-or-nothing.	20 simultaneous assignments (TC-23) and rollback test (TC-24).
SC-4	**Accountability.** Every state change and key security event is recorded with who, when and what, and the application cannot alter or delete these records.	Audit tests TC-38 to TC-40.
SC-5	**Verified quality.** Every Must Have requirement has passing automated tests, the performance targets (NFR-06) are met, and a reviewer can start the system from a clean clone with three documented commands.	Section 7 results; NT-04, NT-07.

3. SOLUTION OVERVIEW AND SCOPE

3.1 Solution Overview

The system is one web application with role-based access. Residents report faults through a form. Supervisors triage each request and assign it to a Technician. Technicians start the job and record what they did. Supervisors verify the work and close the request. Wardens watch their own residence on a dashboard that cannot change anything, and can report a fault for a Resident who cannot reach their account, and Admins create and manage user accounts and read the audit log. Every request follows the single lifecycle in Section 5.1.

The system boundary is drawn around the web client, the API, the database and the login and access-control code. The five roles are outside it and reach it through a browser. There are no external systems: no university login service, no finance system, and no email, SMS or

WhatsApp gateway. The database is inside the boundary, so it does not appear as an external entity in Figure 3.1.

The system is not meant for emergencies. The report form always shows a notice that anything dangerous must go to campus security directly.

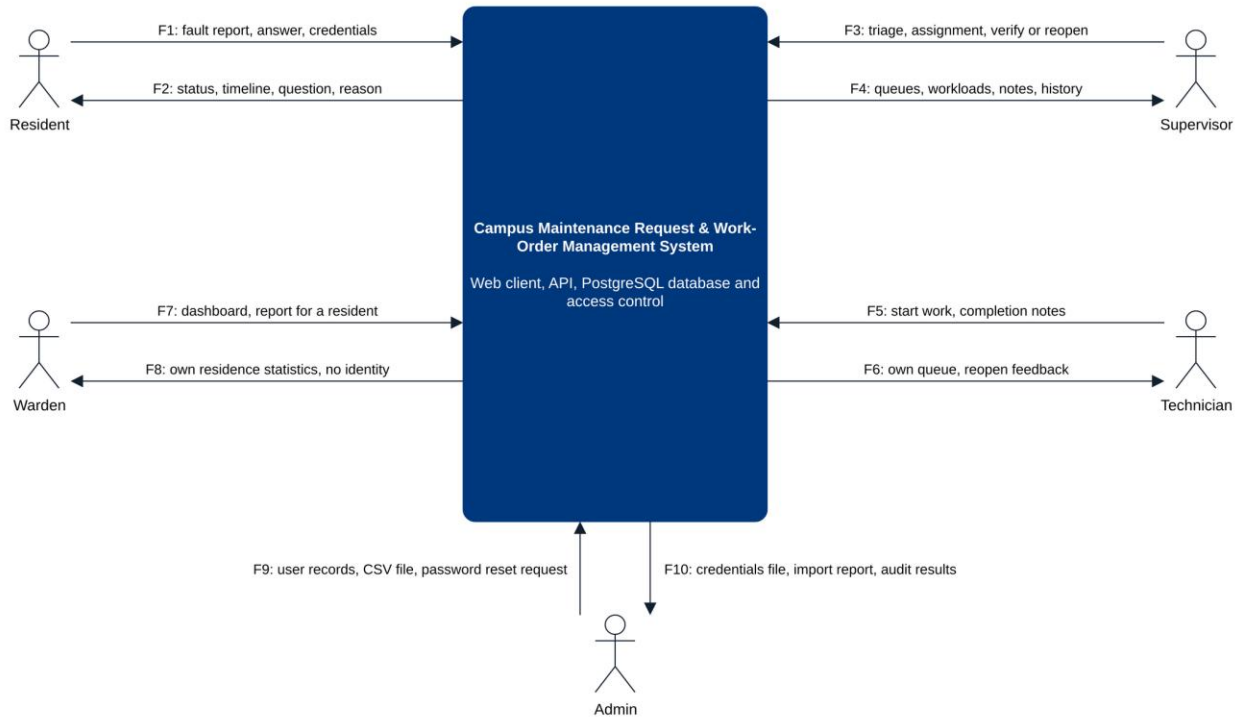


Figure 3.1: System context diagram

Table 3.1 lists what the arrows in Figure 3.1 carry.

Flow	From	To	Data
F1	Resident	System	Fault report (location, category, description), clarification answer, credentials
F2	System	Resident	Request status and timeline, clarification question, rejection reason
F3	Supervisor	System	Triage decision, assignment or reassignment, verify or reopen
F4	System	Supervisor	Triage queue, Technician workloads, completion notes, dashboard, request history
F5	Technician	System	Start work, completion notes
F6	System	Technician	Own work queue (no resident identity), reopen feedback
F7	Warden	System	Dashboard request, fault report for a Resident, clarification answer on that report
F8	System	Warden	Statistics for their own residence (no resident identity), status of their own reports
F9	Admin	System	User records, CSV file, password reset request
F10	System	Admin	Temporary credentials file, import report, audit search results

3.2 Final Release Scope

The final release has the capabilities below. The minimum viable product is the set of Must Have requirements in Section 4.1, and the Should Have requirements are included as well. If anything listed here is not working at submission, it must be reported in Section 3.4.

- Sign-in, sign-out, password management and role-based access for five roles (FR-01, FR-02).
- User administration and bulk account creation from a CSV file (FR-03, FR-04).
- Fault reporting by Residents, or by Wardens for a Resident, with a validated location and category, and protection against double submission (FR-05).
- Residents, and Wardens reporting for them, following their requests and answering questions (FR-06, FR-07).
- Supervisor triage, assignment, reassignment, verification and reopening (FR-08 to FR-12).
- Technician work queue, starting work and completing it with mandatory notes (FR-13 to FR-15).
- A read-only dashboard for Wardens and Supervisors (FR-16).
- An append-only audit trail with request history and Admin search (FR-17, FR-18).

3.3 Excluded and Deferred Capabilities

I left these out on purpose. Items that are functional requirements carry their FR number and appear in Section 4.1 as Deferred or Excluded.

Capability	Ref	Status	Reason	What covers it in this release
Email, SMS and WhatsApp notifications	FR-21	Deferred	Needs an outside provider, handling of failed delivery and consent. It is separate from lifecycle integrity.	“Action required” counts on the dashboards. A notifier could later read the audit events.
Photo attachments	FR-23	Excluded	Adds file storage, content checks and more personal information, and no success criterion depends on it.	Location, category and a written description.
Cancel own request	FR-19	Deferred	Adds a state and transitions without helping a success criterion.	A Supervisor rejects a duplicate or mistaken request with a reason.
Master-data screens	FR-20	Deferred	Residences, locations and categories change rarely.	Loaded by the seed command.
Safety flag and automatic escalation	FR-22	Deferred	Needs service-level times that the case study does not give.	Emergency notice on the form; dashboards show request age.

Capability	Ref	Status	Reason	What covers it in this release
Single sign-on, multi-factor sign-in	—	Deferred	Depends on university identity services I do not control.	Own accounts, Argon2id hashing, sign-in throttling.
Cost, procurement and stock tracking	—	Excluded	The system tracks the work, not what it costs.	None needed.
Native mobile app, offline mode	—	Excluded	A responsive web client meets the need.	Works at 360 px width (NFR-07).
AI/ML, IoT, GPS, predictive maintenance	—	Excluded	Not in the proposal and no data or evidence to support them.	None needed.

3.4 Known Limitations and Open Issues

These are the limitations of the submitted system. Defects found during the work are in Section 7.4.

ID	Limitation	Impact	Status and workaround
LIM-01	No push notifications.	Users find out about changes only when they open the application.	Accepted. “Action required” on My Requests and counts on the Supervisor queues.
LIM-02	An Admin hands temporary credentials to users by hand.	A credential could go to the wrong person before first use.	Accepted. The first sign-in forces a password change, and issuing and resetting are audited.
LIM-03	The audit log is append-only for the application and its database role, but a database superuser or schema owner can still change it. There is no cryptographic chaining.	Tamper-resistant, not tamper-proof against a privileged insider.	Accepted. I call it “append-only”, never “immutable”.
LIM-04	One host, manual backups, no high availability.	A host failure means downtime and loss of data since the last backup.	Accepted. Backup and restore steps are in Section 8.
LIM-05	Performance is checked only up to 30 concurrent users and 5,000 requests. These are my assumptions, not measurements from the residence.	Behaviour beyond that is unknown.	Accepted. Results are reported as measured.
LIM-06	No automated retention or data-subject tools.	Retention and deletion are manual decisions for the institution.	Accepted. Records are never hard-deleted.
LIM-07	Technicians cannot see who reported a fault or contact the Resident through the system.	Access to rooms is arranged through the Supervisor.	Chosen to limit personal data (BR-04).

ID	Limitation	Impact	Status and workaround
LIM-08	The evidence comes from one residence.	Fit for other residences is unchecked.	Accepted. I make claims only about Moroka Hall.
LIM-09	A request in Needs Clarification never expires by itself.	It can stay open if the Resident never answers.	Accepted. A Supervisor can reject it with a reason.
LIM-10	The assessed system runs in local containers over HTTP on localhost.	No TLS is configured, so this profile is for local evaluation only.	Accepted. A public deployment would need TLS and HSTS at the proxy first.
LIM-11	Rooms come from the institution's list (414 rooms). Common areas and fault categories are my own list, not a survey of the residence.	Some real common areas may be missing.	Accepted. The seed file can be extended.
LIM-12	Through Docker on one machine, every browser reaches the API from one IP address.	Five failed sign-ins pause all sign-ins from that machine for up to a minute.	Accepted. This is the rule in NFR-02, and the pause clears by itself.

Status at submission: every Included feature was delivered and tested. None is incomplete.

4. SYSTEM REQUIREMENTS SPECIFICATION

4.1 Functional Requirements

Every requirement has a permanent ID, and priority uses MoSCoW. “Included” means the feature is part of the system that is submitted and assessed. Deferred and Excluded items are listed so that this table matches Section 3.3.

ID	Requirement	Description	Priority	Release status
Accounts and access				
FR-01	Authenticate	Users shall sign in with email and password and sign out. A failed sign-in shall return the same generic error whichever of the two was wrong, and repeated failures shall be throttled.	Must Have	Included
FR-02	Manage password	A user who signs in with an Admin-issued temporary password shall be required to set their own password before using the system. Any user shall be able to change their own password, subject to a password policy (at least 10 characters, not a commonly used password).	Must Have	Included

ID	Requirement	Description	Priority	Release status
FR-03	User administration	An Admin shall create users, edit their role and residence, deactivate and reactivate them, and issue a new temporary password. The system shall refuse to deactivate a Technician who has active work orders and shall refuse to remove the last active Admin.	Must Have	Included
FR-04	Bulk provisioning	An Admin shall create many accounts from one CSV file (email, full_name, role, residence, student_number, and optionally room). The whole file shall be validated first. Either every account is created or none is, and every invalid row is reported.	Should Have	Included
Resident				
FR-05	Submit request	A Resident shall submit a fault by choosing their assigned room or a common area in their own residence and a Fault Category and entering a description of 10 to 2,000 characters. The system shall store it as Submitted, return a tracking ID and prevent accidental duplicate submission. A Warden may do the same for any Location in their residence, for a Resident who cannot reach their account.	Must Have	Included
FR-06	Track requests	A Resident shall see their own requests with current status, and for each a chronological timeline of changes and notes. Requests waiting for the Resident's answer shall be listed under “Action required”.	Must Have	Included
FR-07	Respond to clarification	The person who reported a fault shall answer a clarification question on it, which returns the request to Submitted.	Must Have	Included
Supervisor				
FR-08	Triage queue	A Supervisor shall see all Submitted requests, oldest first, filterable by residence and category.	Must Have	Included
FR-09	Triage decision	A Supervisor shall approve a Submitted request and set its priority (Low, Medium, High or Critical), reject a Submitted or Approved request with a written reason, or return a Submitted request to the Resident with a clarification question.	Must Have	Included
FR-10	Assign work order	A Supervisor shall assign an Approved request to exactly one active Technician, which creates a work order. The assignment list shall show each Technician's current number of open work orders.	Must Have	Included

ID	Requirement	Description	Priority	Release status
FR-11	Reassign work order	A Supervisor shall reassign a request that is Assigned, In Progress or Reopened to a different active Technician, giving a written reason. The previous assignment ends and a new one begins.	Should Have	Included
FR-12	Verify, close or reopen	A Supervisor shall review the completion notes of a Work Completed request and either verify and close it or reopen it with written feedback.	Must Have	Included
Technician				
FR-13	Work queue	A Technician shall see only requests currently assigned to them, ordered by priority and then assignment time, without any resident identity.	Must Have	Included
FR-14	Start work	A Technician shall start an Assigned or Reopened work order, which moves the request to In Progress.	Must Have	Included
FR-15	Complete work	A Technician shall complete an In Progress work order by entering completion notes of at least 20 characters after trimming. Without valid notes the transition shall be refused.	Must Have	Included
Oversight and audit				
FR-16	Operational dashboard	The system shall provide a read-only dashboard showing counts by status, priority and category, the age of open requests and the mean time to closure. A Warden sees only their own residence; a Supervisor sees all residences.	Should Have	Included
FR-17	Audit recording	Together with the change itself, the system shall record an audit event for every request state transition and assignment change and for key security events (failed sign-in, role change, deactivation, password reset, bulk import).	Must Have	Included
FR-18	Audit review	A Supervisor shall see the full event history of any request. An Admin shall search the audit log by actor, action, request and date range.	Should Have	Included
Not in this release				
FR-19	Cancel own request	A Resident shall cancel their own request before it is approved.	Could Have	Deferred
FR-20	Master-data screens	An Admin shall maintain residences, locations and fault categories through the application. In this release this data is loaded by a seed command.	Could Have	Deferred
FR-21	Notifications	The system shall notify users by email, SMS or WhatsApp when a request changes state.	Won't Have	Deferred

ID	Requirement	Description	Priority	Release status
FR-22	Safety flag and escalation	A Resident shall flag a request as a safety hazard and unattended requests shall escalate automatically after a defined period.	Could Have	Deferred
FR-23	Photo attachments	A Resident shall attach photographs to a request.	Won't Have	Excluded

4.2 Non-Functional Requirements

Each requirement has a target I can measure and a way to check it. The load figures are my own engineering assumptions, not service levels agreed with the residence (LIM-05).

ID	Quality attribute	Requirement	Verification
NFR-01	Security: authorisation	Every endpoint rejects unauthenticated calls (401) and calls by a role not permitted (403). An object outside the caller's scope is reported as not found (404). Target: zero successful unauthorised calls.	Authorisation matrix test (NT-01, ST-01, ST-03, ST-04)
NFR-02	Security: credentials and sessions	Passwords are stored only as salted Argon2id hashes. Session cookies are HttpOnly and SameSite=Lax, and state-changing requests require a CSRF token. After 5 failed sign-ins in one minute for an account or an IP address, further attempts return 429. Sessions expire after 8 hours and sign-out ends the session on the server.	Database and cookie inspection; throttle and session tests (TC-03, NT-02, ST-05)
NFR-03	Reliability: atomicity	The changes made by one transition (request state, work order, audit event) are committed together or not at all. After any sequence of operations the invariants in Section 5.6 hold.	Fault-injection and randomised-sequence tests (TC-24, NT-03)
NFR-04	Reliability: concurrency	When 20 or more clients attempt the same conflicting action on one request simultaneously, exactly one succeeds and the others receive 409. A request never has more than one active work order.	Multi-threaded test on PostgreSQL, 50 repetitions (TC-23)
NFR-05	Auditability	UPDATE, DELETE and TRUNCATE on the audit table fail for the application's database role.	Database test run as that role (TC-39)
NFR-06	Performance	In the evaluation environment (Docker Compose on the reference machine in Section 8), with 5,000 requests seeded and 30 concurrent simulated users, 95% of API requests complete within 1 second and the first load of each dashboard completes within 3 seconds.	Load test (Locust) (NT-04)

ID	Quality attribute	Requirement	Verification
NFR-07	Usability and compatibility	The Resident and Technician screens work at a 360 px viewport without horizontal scrolling and on the latest Chrome and Edge. Every form field has a programmatic label. Lighthouse accessibility score is at least 90 on SCR-01, 04, 06, 09 and 10.	Viewport, browser and Lighthouse checks (NT-05)
NFR-08	Data integrity	Referential, domain and uniqueness rules are enforced by the database (foreign keys, CHECK constraints, unique and partial unique indexes) and not only by application code.	Direct SQL violation tests (TC-25, NT-06)
NFR-09	Reproducibility	From a clean clone, the system builds, migrates, seeds and starts with at most three documented commands, and the automated test suite passes in a clean environment.	Clean-environment run in CI (NT-07)
NFR-10	Maintainability	Backend code is layered (API, service, data). No business rule is implemented in a view or serialiser, and request status is assigned only by the transition service. Lint and TypeScript type checks pass.	CI checks and repository check (NT-08)
NFR-11	Security baseline	Responses carry Content-Security-Policy, X-Content-Type-Options, Referrer-Policy and framing-protection headers. Dependency audits of production packages report no unaddressed High or Critical vulnerability at submission.	Header test; pip-audit and npm audit (NT-09, ST-01, ST-02)

4.3 Business Rules

Business rules have IDs so that requirements, diagrams and tests can point to them.

ID	Rule	Applies to	Verified by
BR-01	**Single active assignment.** A request has at most one active work order, and therefore one accountable Technician, at any time.	FR-10, FR-11	TC-23, TC-25
BR-02	**Mandatory narrative.** A written note is mandatory for a rejection, a clarification question, a clarification answer, a reassignment and a reopening (at least 10 characters each) and for completion notes (at least 20 characters). Text is trimmed first, so whitespace-only input counts as empty.	FR-07, FR-09, FR-11, FR-12, FR-15	TC-15, TC-18, TC-30, TC-33

ID	Rule	Applies to	Verified by
BR-03	**Supervisor reassignment authority.** Only a Supervisor may change the Technician on an assigned request, including one In Progress. The old work order ends with reason REASSIGNED, a new one is created for a different active Technician, the request returns to Assigned, and the audit event records both Technicians and the reason.	FR-11	TC-26
BR-04	**Read-only oversight and data minimisation.** A Warden sees only their own residence, and their only writes are reporting a fault for a Resident and answering questions on that report. Neither Wardens nor Technicians receive resident identity in any response.	FR-13, FR-16	TC-28, TC-36, ST-06
BR-05	**Ownership.** A Resident can see and act on only their own requests.	FR-05 to FR-07	TC-14, ST-04
BR-06	**Terminal states are final.** Rejected and Verified & Closed are final. A recurrence of a closed fault is a new request; rework before closure is done through Reopened.	FR-09, FR-12	TC-34
BR-07	**Active accounts only.** Only active users can sign in or be assigned work. Deactivation ends the user's sessions. A Technician with active work orders cannot be deactivated, and at least one active Admin must always exist.	FR-01, FR-03, FR-10	TC-04, TC-06, ST-05
BR-08	**Priority.** Priority is set only by a Supervisor, at approval.	FR-09	TC-17, ST-03
BR-09	**Location and category scope.** A Resident may report only against their assigned room and active common areas in their own residence. A Warden may report against any active Location in their own residence. Both use active Fault Categories.	FR-05	TC-11
BR-10	**Separation of duties.** Only a Supervisor may verify and close or reopen a request, so the Technician who completed the work cannot approve it.	FR-12	TC-34

4.4 Acceptance Criteria for Included Requirements

Acceptance criteria use given/when/then wording, and their IDs are what Section 7 refers to. I wrote them for the main Included requirements; the rest are covered by the tests in Section 7. Each set has success and failure cases.

ID	Criterion
FR-01 Authenticate	
AC-01.1	Given an active user, when valid credentials are submitted, then a session is created and the user reaches the home screen for their role.
AC-01.2	Given an unknown email or a wrong password, then the response is 401 with the same message in both cases and a failed-sign-in event is recorded.

ID	Criterion
AC-01.3	Given 5 failed attempts within one minute for one account or one IP address, then the next attempt returns 429 until the window passes.
AC-01.4	Given a deactivated user, then sign-in is refused with the generic message, and deactivation ends any open session.
FR-04 Bulk provisioning	
AC-04.1	Given a valid CSV of N rows, then N accounts are created in one transaction and the Admin receives a one-time file of temporary credentials.
AC-04.2	Given any invalid row, or an email duplicated within the file or already registered (compared case-insensitively), or an unknown role, residence or room, then no account is created and every invalid row is listed with its row number and reason.
AC-04.3	Given a file over 2 MB, over 2,000 rows or not UTF-8 CSV, then it is rejected before rows are processed. Given a non-Admin caller, then the response is 403.
FR-05 Submit request	
AC-05.1	Given a signed-in Resident, a Location in their residence, a Fault Category and a description of 10 to 2,000 characters, then the request is stored as Submitted, the tracking ID is returned and an audit event is recorded.
AC-05.2	Given a missing, shorter than 10 or longer than 2,000 character description, or a missing, inactive or other-residence Location or Category, then the response is 422 with field-level errors and nothing is stored.
AC-05.3	Given the same client submission token sent twice, then the second call returns the original request and no duplicate is created.
AC-05.4	Given a caller who is not a Resident or a Warden, then the response is 403. The form always displays the notice that emergencies go to campus security.
FR-09 Triage decision	
AC-09.1	Given a Submitted request, approval with a priority sets Approved. Approval without a priority returns 422.
AC-09.2	Given a Submitted or Approved request, rejection with a reason of at least 10 characters sets Rejected and the reason is visible to the Resident. Without a reason the response is 422.
AC-09.3	Given a Submitted request, a clarification question sets Needs Clarification and is visible to the Resident.
AC-09.4	Given two Supervisors acting on the same request at the same time (for example approve and reject), then exactly one succeeds and the other receives 409 with the current status. A decision sent with a stale expected status also returns 409 and changes nothing.
FR-10 Assign work order	
AC-10.1	Given an Approved request and an active Technician, then a work order is created, the request becomes Assigned and an event is written, all in one transaction, and the request appears in that Technician's queue.
AC-10.2	Given an inactive user or a user who is not a Technician, then the response is 422 and nothing changes.
AC-10.3	Given a failure injected after the work order is created and before commit, then the whole assignment is rolled back.
AC-10.4	Given simultaneous assignments of one request, then exactly one succeeds, the others receive 409, and exactly one active work order exists.

ID	Criterion
FR-11 Reassign work order	
AC-11.1	Given an Assigned, In Progress or Reopened request, a different active Technician and a reason of at least 10 characters, then the old work order ends with reason REASSIGNED, a new one is created, the request becomes Assigned and the event records both Technicians.
AC-11.2	Given the same Technician or an inactive user, then the response is 422. Given any other request state, then the response is 409.
AC-11.3	Given the previous Technician, then the request has left their queue and any further action by them returns 404.
FR-12 Verify, close or reopen	
AC-12.1	Given a Work Completed request, verification sets Verified & Closed, records the closing time and ends the work order with reason COMPLETED.
AC-12.2	Given a Work Completed request, reopening with feedback of at least 10 characters sets Reopened and the feedback is visible to the Technician. Without feedback the response is 422.
AC-12.3	Given a request that is not Work Completed, then verification or reopening returns 409.
FR-15 Complete work	
AC-15.1	Given notes of at least 20 characters after trimming, then the request becomes Work Completed and leaves the Technician's active queue.
AC-15.2	Given empty, whitespace-only or shorter notes, then the response is 422 and the state is unchanged, in the client and on the server.
AC-15.3	Given a double submission, then exactly one transition occurs and the second call returns 409.
FR-16 Operational dashboard	
AC-16.1	Given a Warden, then only their residence's data is returned, resident identity is absent and every write returns 403 except reporting a fault and answering questions on their own reports.
AC-16.2	Then aggregate figures equal those obtained by direct SQL over the seeded data.
AC-16.3	Given a Supervisor, then data for all residences is available.
FR-17 Audit recording	
AC-17.1	Every successful transition writes exactly one event with actor, UTC timestamp, action, previous state and new state. A refused transition writes no transition event.
AC-17.2	UPDATE, DELETE and TRUNCATE on the audit table fail when run with the application's database role.
AC-17.3	Failed sign-in, role change, deactivation, password reset and bulk import each produce an event.

4.5 Requirements Traceability Matrix

This table follows each Included requirement from the problem it answers to its tests: evidence (P or SC), use case, screen, class or service, database table, business rule and test. Use case numbers match requirement numbers, and test IDs refer to Section 7.

FR	Evidence	Use case	Screen	Class / service	Tables	Rules	Tests
FR-01	SC-2	UC-01	SCR-01	AuthService, PermissionPolicy	users, django_session, audit_logs	BR-07; NFR-02	TC-01 to 04
FR-02	SC-2	UC-02	SCR-02	AuthService	users	NFR-02	TC-05
FR-03	SC-2	UC-03	SCR-11	UserAdminService	users, audit_logs	BR-07	TC-06
FR-04	SC-2	UC-04	SCR-12	UserImportService, UserAdminService	users, audit_logs	BR-07; NFR-03	TC-07 to 09
FR-05	P-1, P-2; SC-1	UC-05	SCR-04	TransitionService, PermissionPolicy	maintenance_requests, locations, fault_categories, audit_logs	BR-05, BR-09	TC-10 to 13, TC-35
FR-06	P-2; SC-1	UC-06	SCR-03, SCR-05	PermissionPolicy (scope), AuditService	maintenance_requests, audit_logs	BR-05	TC-14
FR-07	P-1; SC-1	UC-07	SCR-05	TransitionService	maintenance_requests, audit_logs	BR-02, BR-05	TC-15, TC-19, TC-35
FR-08	P-1, P-3; SC-1	UC-08	SCR-06	PermissionPolicy (scope)	maintenance_requests	—	TC-16
FR-09	P-1, P-3; SC-1	UC-09	SCR-06	TransitionService, TransitionRule	maintenance_requests, audit_logs	BR-02, BR-08	TC-17 to 20, TC-35
FR-10	P-1, P-3; SC-3	UC-10	SCR-07	TransitionService	work_orders, maintenance_requests, audit_logs	BR-01; NFR-04	TC-21 to 25, TC-35
FR-11	P-3; SC-3	UC-11	SCR-07	TransitionService	work_orders, maintenance_requests, audit_logs	BR-01, BR-02, BR-03	TC-26, TC-27, TC-35
FR-12	P-3; SC-1	UC-12	SCR-08	TransitionService	work_orders, maintenance_requests, audit_logs	BR-02, BR-06, BR-10	TC-32 to 35
FR-13	P-3; SC-2	UC-13	SCR-09	PermissionPolicy (scope)	work_orders, maintenance_requests	BR-04	TC-28
FR-14	P-3; SC-1	UC-14	SCR-09	TransitionService	maintenance_requests, audit_logs	BR-01	TC-29, TC-35
FR-15	P-3; SC-1	UC-15	SCR-09	TransitionService	maintenance_requests, audit_logs	BR-02	TC-30, TC-31, TC-35
FR-16	P-4; SC-2	UC-16	SCR-10	ReportingService, PermissionPolicy	maintenance_requests, work_orders	BR-04; NFR-06	TC-36, TC-37

FR	Evidence	Use case	Screen	Class / service	Tables	Rules	Tests
FR-17	P-4; SC-4	(system behaviour)	—	AuditService, TransitionService	audit_logs	NFR-03, NFR-05	TC-38 to 40
FR-18	P-4; SC-4	UC-18	SCR-05, SCR-13	AuditService, PermissionPolicy	audit_logs	NFR-05	TC-41

Classes and services are in Section 5.4, tables in Section 5.6 and screens in Section 5.5. FR-19 to FR-23 are not traced because they are not built.

5. SYSTEM DESIGN SPECIFICATION

5.1 Use Cases and Workflows

Use case diagram

Figure 5.1 shows the five roles and what each can do in the final release. Use case numbers match requirement numbers, so UC-05 is FR-05. FR-17 (audit recording) is something the system does by itself, so it has no use case and shows up in the sequence diagrams instead.

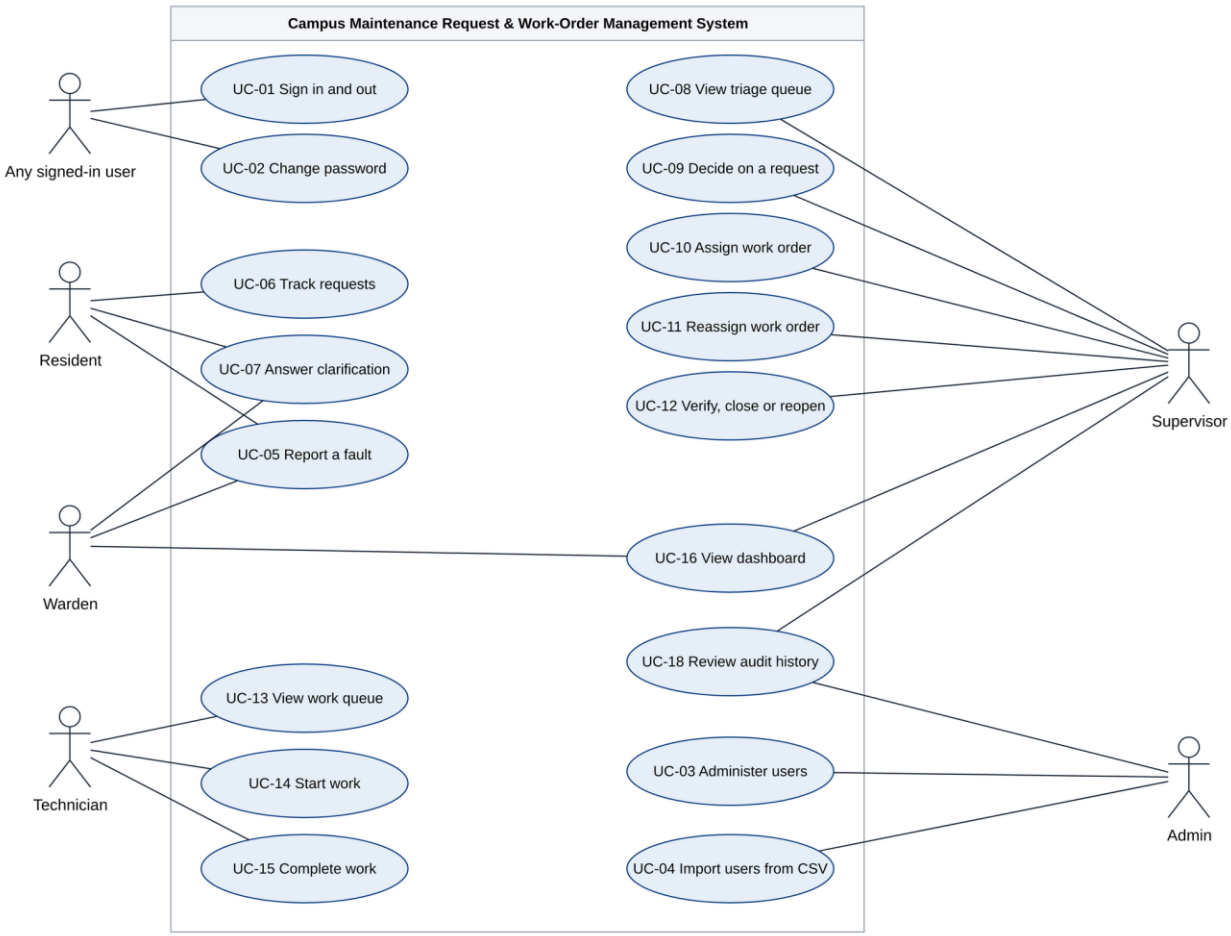


Figure 5.1: Use case diagram

Workflow 1: report and triage

This workflow covers UC-05 to UC-09. A resident reports a fault. A supervisor approves it, rejects it or asks a question, and the resident can answer. Figure 5.2 includes the failure paths: invalid input, a repeated submit, and two supervisors acting on the same request. Figure 5.3 shows the same flow as calls between the browser, the API, the transition service and the database.

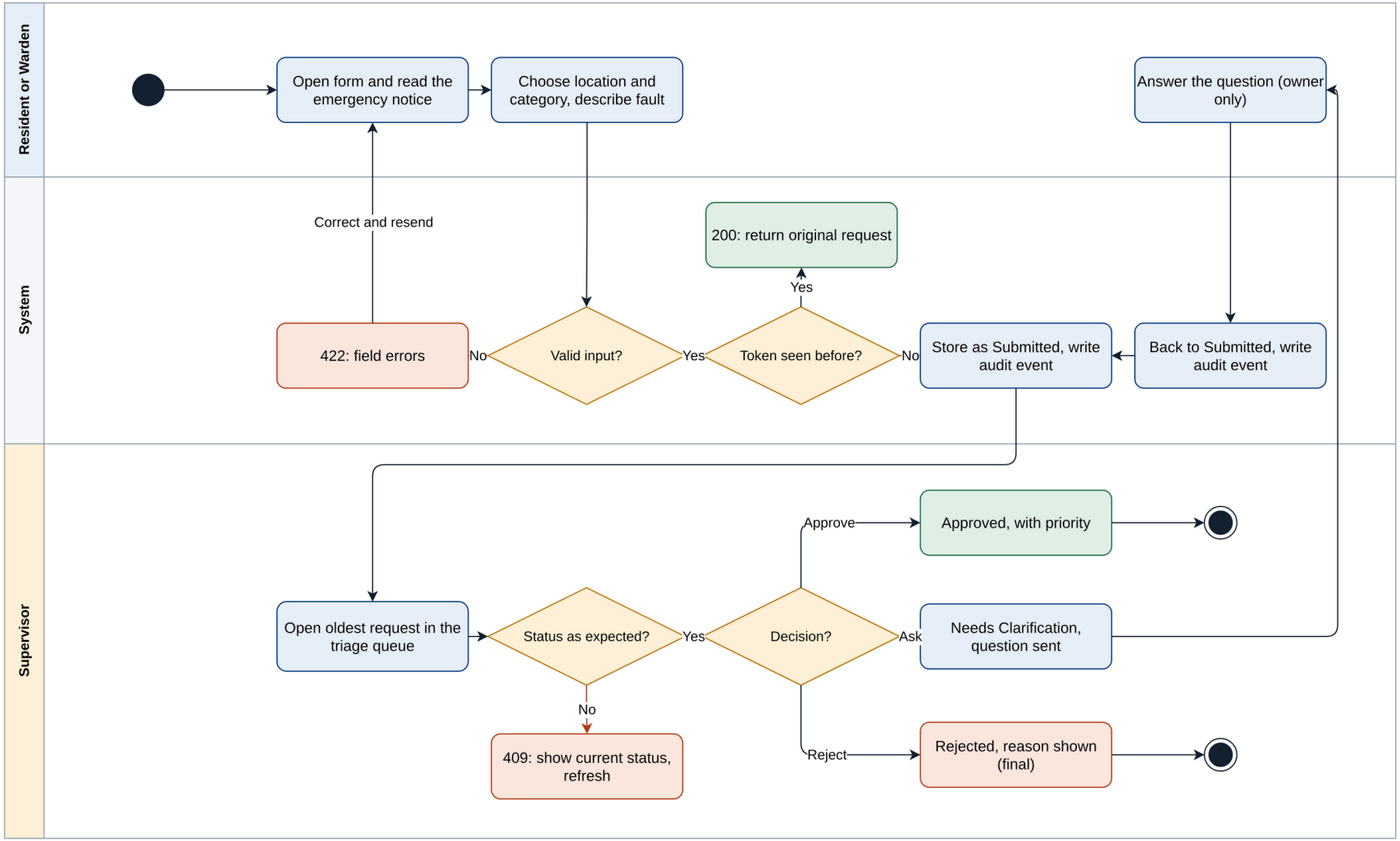


Figure 5.2: Activity diagram, Workflow 1: report and triage

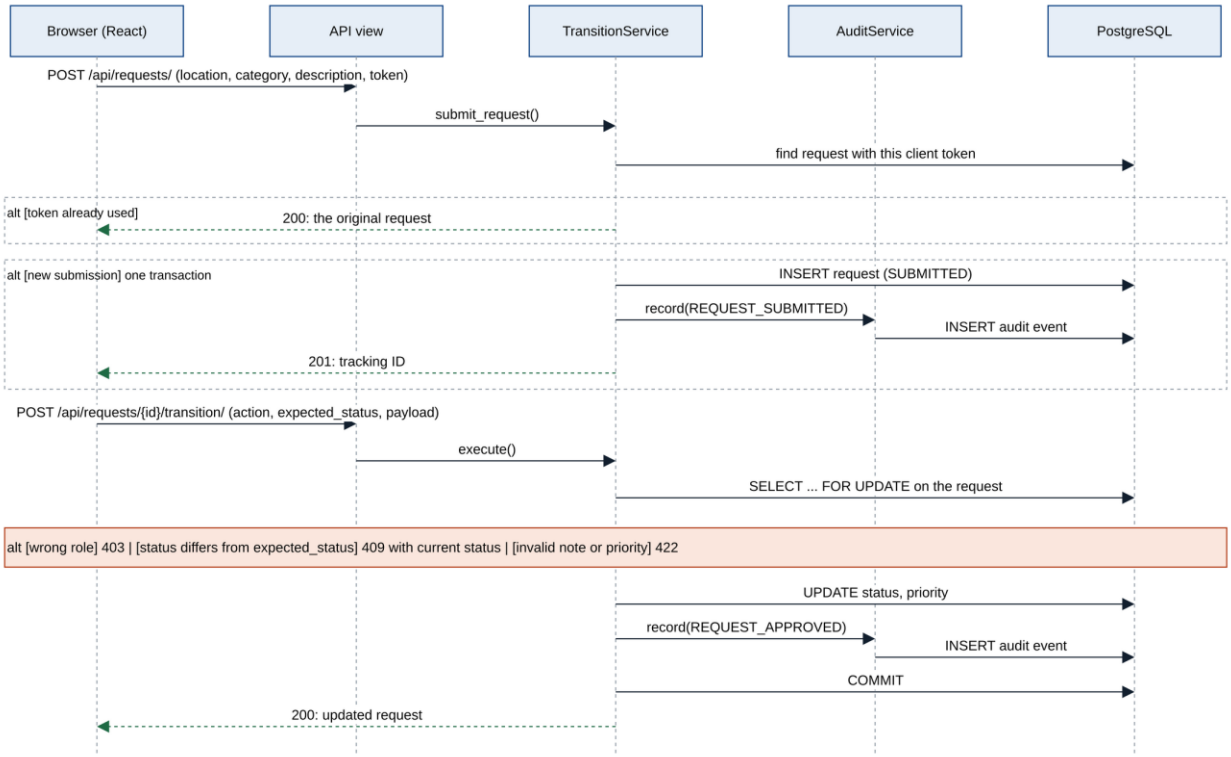


Figure 5.3: Sequence diagram, Workflow 1: submit and triage decision

Workflow 2: assign, work and verify

This workflow covers UC-10 to UC-15. Figure 5.4 shows assigning and reassigning, completing work with and without notes, and the reopen loop. Figure 5.5 shows the locking that stops two supervisors assigning the same request, and the transaction that completes a job.

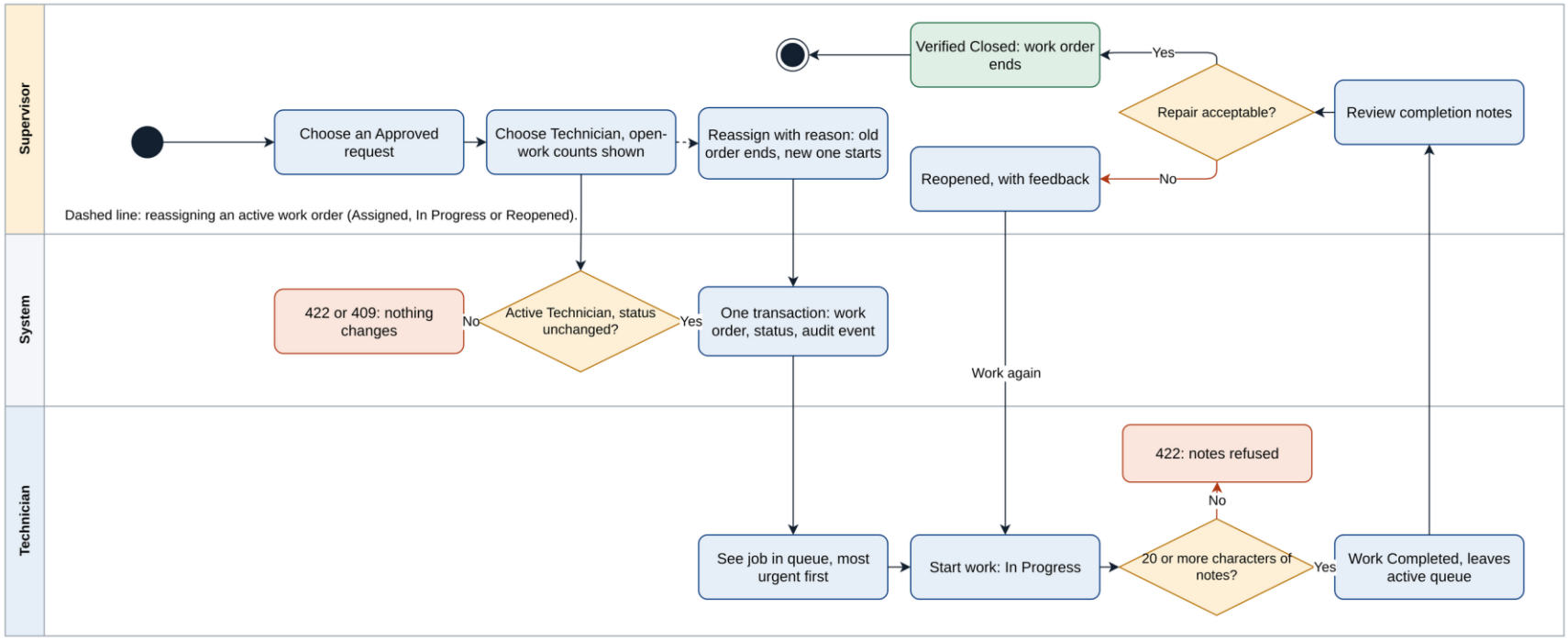


Figure 5.4: Activity diagram, Workflow 2: assign, work and verify

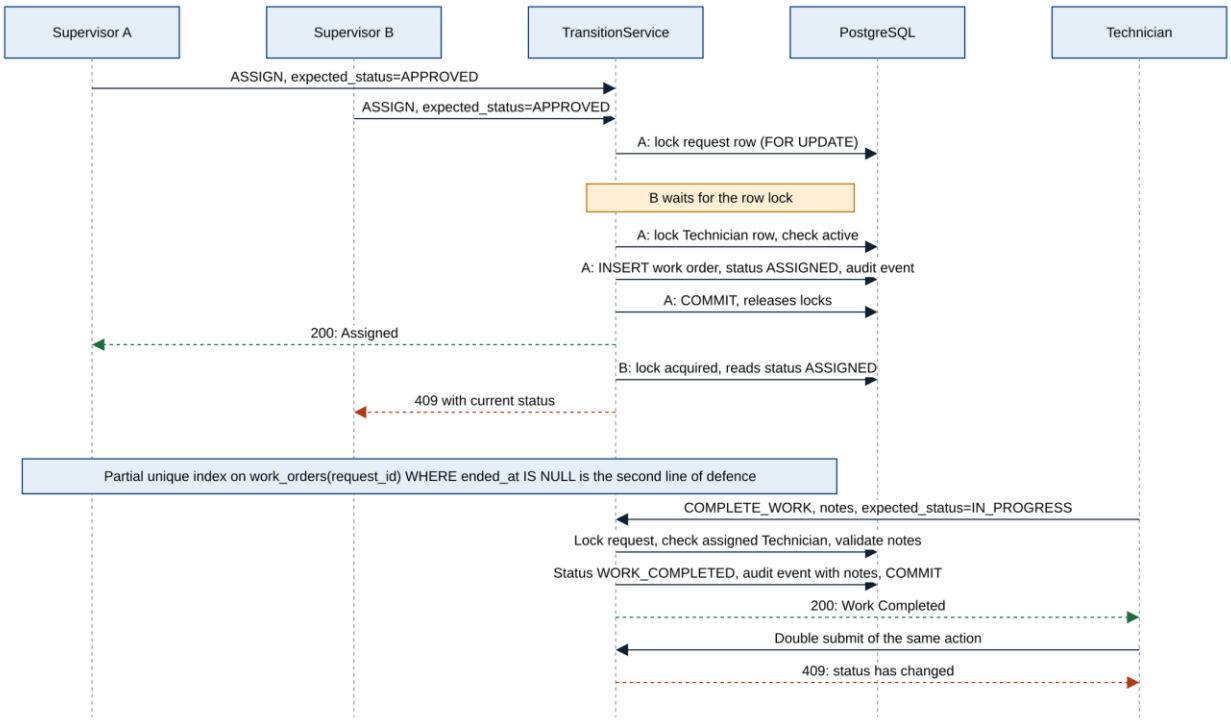
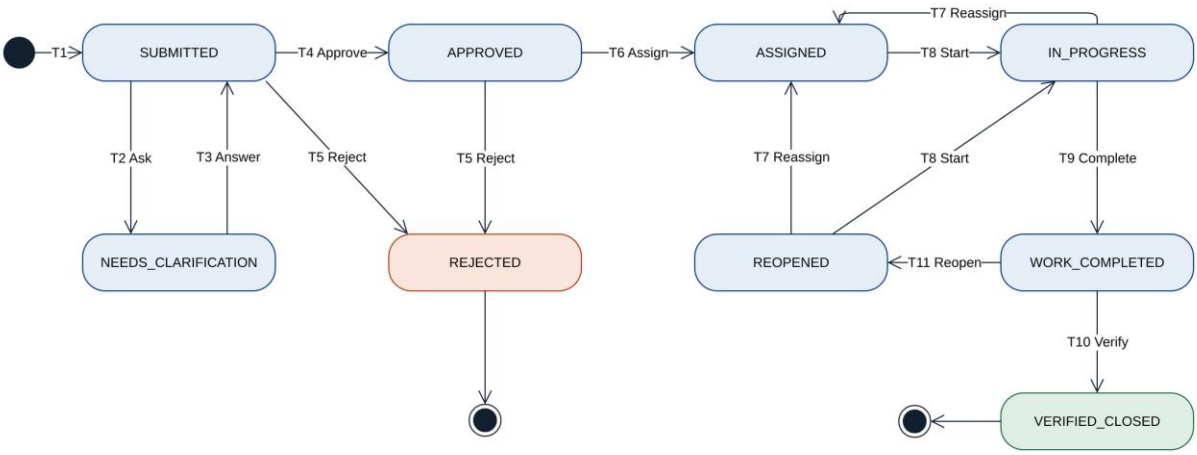


Figure 5.5: Sequence diagram, Workflow 2: concurrent assignment and completion

State diagram and transition table

A request has one status, and that is the only lifecycle state in the system. Work orders record who is assigned but have no status of their own (CH-05). Table 5.1 is the full list of legal transitions. The transition service is driven by that table, and Figure 5.6 is a picture of it. Anything not in the table is refused with HTTP 409.



T7 Reassign also applies while ASSIGNED: the request stays ASSIGNED with a different Technician. Anything not shown is refused with 409. REJECTED and VERIFIED_CLOSED are final.

Figure 5.6: State diagram of a maintenance request

Table 5.1: Request lifecycle, legal transitions. “Owner” is the person who submitted the request. “Assigned Technician” is the Technician on the active work order.

#	Action	From → To	Actor	Guard and required input	Audit action
T1	SUBMIT	(none) → SUBMITTED	Resident or Warden	Valid location and category for the reporter (BR-09); description 10 to 2,000 characters	REQUEST_SUBMITTED
T2	REQUEST_CLARIFICATION	SUBMITTED → NEEDS_CLARIFICATION	Supervisor	Question, at least 10 characters	CLARIFICATION_REQUESTED
T3	PROVIDE_CLARIFICATION	NEEDS_CLARIFICATION → SUBMITTED	Owner	Answer, at least 10 characters	CLARIFICATION_PROVIDED
T4	APPROVE	SUBMITTED → APPROVED	Supervisor	Priority (Low, Medium, High, Critical)	REQUEST_APPROVED
T5	REJECT	SUBMITTED or APPROVED → REJECTED	Supervisor	Reason, at least 10 characters	REQUEST_REJECTED
T6	ASSIGN	APPROVED → ASSIGNED	Supervisor	Active Technician; creates the work order	WORK_ASSIGNED
T7	REASSIGN	ASSIGNED, IN_PROGRESS or REOPENED → ASSIGNED	Supervisor	Different active Technician; reason of at least 10 characters; ends old work order and creates a new one	WORK_REASSIGNED
T8	START_WORK	ASSIGNED or REOPENED → IN_PROGRESS	Assigned Technician	None	WORK_STARTED
T9	COMPLETE_WORK	IN_PROGRESS → WORK_COMPLETED	Assigned Technician	Completion notes, at least 20 characters	WORK_COMPLETED
T10	VERIFY_CLOSE	WORK_COMPLETED → VERIFIED_CLOSED	Supervisor	Ends the work order (reason COMPLETED); sets closed_at	REQUEST_VERIFIED_CLOSED
T11	REOPEN	WORK_COMPLETED → REOPENED	Supervisor	Feedback, at least 10 characters	REQUEST_REOPENED

REJECTED and VERIFIED_CLOSED are final (BR-06). Every transition request carries the status the caller last saw (`expected_status`). The server compares it with the locked row and answers 409 with the current status if they differ.

5.2 System Architecture and Technology Stack

The system is a modular monolith: one Django codebase and one deployable unit, split into apps by responsibility, with PostgreSQL as the only place data is kept. Figure 5.7 shows only components that really exist.

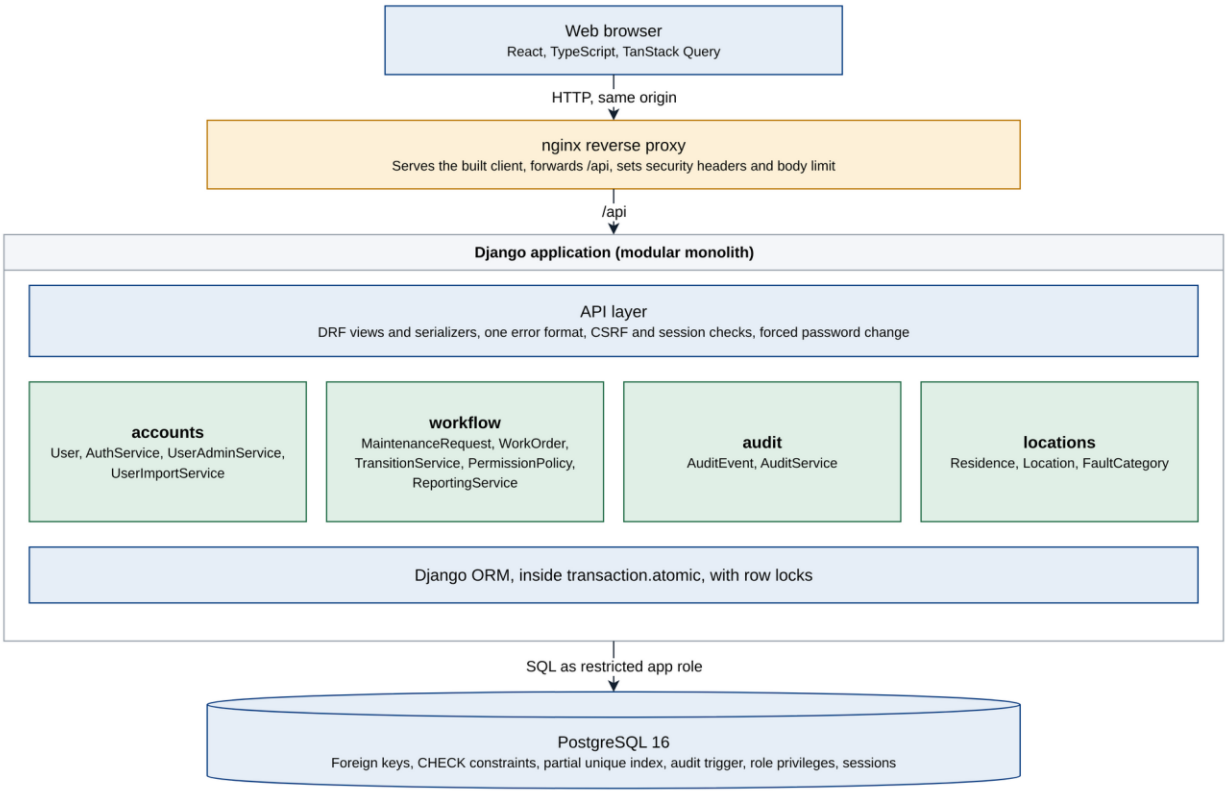


Figure 5.7: Logical architecture

Table 5.2: Components and technology stack

Component	Technology	Responsibility
Web client	React, TypeScript, Vite, React Router, TanStack Query, React Hook Form with Zod	Role-specific screens that work on a phone. Client-side validation for convenience only. Caches server data. Request and response types are written in src/types; src/api/schema.d.ts is generated from the OpenAPI schema for reference.
Reverse proxy	nginx	Serves the built client and forwards `/api` to the backend on one origin. Sets security headers and request-size limits.
API layer	Django REST Framework, drf-spectacular	Authenticates, authorises, checks the shape of input and turns errors into the error codes in Section 6.2. Holds no business rules.
Service layer	Python 3.12; apps accounts, locations, workflow, audit	TransitionService, PermissionPolicy, AuditService, AuthService, UserAdminService, UserImportService and ReportingService (Section 5.4).
Data layer	Django ORM and migrations, PostgreSQL 16	System of record. Foreign keys, row locks, the partial unique index, CHECK constraints, the audit trigger and role privileges.
Authentication	Django sessions, Argon2id (argon2-cffi)	Server-side sessions in HttpOnly cookies, password hashing and CSRF protection.

Component	Technology	Responsibility
Packaging	Docker, Docker Compose	Repeatable build and start. Keeps the database off the host network.
Quality tools	pytest, Vitest, Playwright, Locust, Ruff, ESLint, GitHub Actions	Unit, integration, end-to-end, load and static checks. Tests run on PostgreSQL so locks and constraints are really used.

Django 5.2 is a long-term-support release and gets security fixes until April 2028 (Django Software Foundation, n.d.-b). Dependency versions are pinned in lock files.

5.3 Key Architecture and Design Decisions

Four decisions shaped the system more than any others. For each I say what I chose, why, and what I turned down.

ADR-01: PostgreSQL as the database

I chose PostgreSQL 16. The data is relational (users, locations, requests, assignments), and a few rules must hold even if my application code has a bug: one active Technician per request, only legal status values, and a history that cannot be edited. PostgreSQL can enforce these itself with foreign keys, CHECK constraints, partial unique indexes, row locks, triggers and role privileges (PostgreSQL Global Development Group, n.d.-a, n.d.-b).

I looked at MySQL, MongoDB and SQLite. MySQL has no partial index, and BR-01 depends on one. MongoDB does support multi-document transactions, so atomicity is not my objection, but it does not enforce references between documents and the dashboard needs joins and totals. SQLite has no row-level locking and allows one writer at a time, so I could not use it to test the concurrency design.

The cost is that I need migrations and every test has to run against a real PostgreSQL instance.

ADR-02: Row locking and one transition service

Two supervisors can press Assign on the same request at the same moment, and a technician can double-click Complete. Either must give one result, not two. Every status change goes through one function, `TransitionService.execute()`. It opens a transaction, locks the request row with `SELECT ... FOR UPDATE`, checks that the status still equals the `expected_status` the browser sent, makes the change, writes the audit event and commits (Django Software Foundation, n.d.-a). The second caller waits a moment, sees the new status and gets a 409 with the current status. As a second line of defence, a partial unique index on

`work_orders(request_id) WHERE ended_at IS NULL` means that even a bug in my code cannot create two active assignments.

I considered checking first and inserting afterwards in application code, but two requests can both see “no work order” and both insert. A unique constraint alone stops a second insert but not other conflicts, such as two reassignments, and its error is hard to explain to a user. Optimistic locking with a version column would work, but every table needs the column and the browser has to retry. Putting everything in one service also means the rules, the notes, the locking and the audit write are enforced in one place. Django signals would not do, because bulk updates and raw SQL skip them.

Each transition holds one row lock briefly. To avoid deadlock I always lock the request first and the Technician second. Deactivating a Technician locks the same Technician row, so a Technician cannot be deactivated and given work at the same instant. The browser has to handle a 409 by refreshing.

ADR-03: Sessions instead of JWT

In Phase 2 I planned to use JWT. I changed to Django's server-side sessions in HttpOnly cookies. A JWT with a role inside stays valid until it expires, so deactivating a user or changing a role would not take effect straight away, and revoking tokens needs a server-side blacklist, which is server state anyway. Tokens kept in browser storage can also be read by injected script. A session is cancelled by deleting one row, and script cannot read the cookie. Django also provides CSRF protection and session rotation. Passwords are hashed with Argon2id, which OWASP lists first for password storage (OWASP Foundation, n.d.).

Single sign-on would suit the university better, but it depends on services I do not control, so it is deferred (Section 3.3). Sessions are stored in PostgreSQL, so no extra service is needed. The client and the API must share one origin, which nginx takes care of, and the client has to send the CSRF header.

ADR-04: One Django application, a React client and Docker Compose

I built one Django application with a React client in front of it, deployed with Docker Compose. The core operation (change a request, create or end a work order, write an audit event) has to be a single database transaction. Splitting that across microservices would make it a distributed transaction for no gain at a few dozen users. I kept the client and the API separate, as the proposal said, so business rules stay out of the user interface and my authorisation tests can call the API directly.

Server-rendered Django pages would have been simpler, and I thought about them. I chose React because there are five role-specific dashboards and I wanted typed contracts between client and server. Docker Compose gives a repeatable setup and keeps the database off the host network. Kubernetes would be far too much for this project.

The costs are two build pipelines and a bigger attack surface in the browser, which I handle with a strict Content-Security-Policy, no raw HTML injection and the CSRF header. There is no high availability (LIM-04, LIM-10).

5.4 Application Class Design

Figure 5.8 shows the domain classes, the enumerations and the service classes that work on them. Domain classes hold data and simple checks, but none of them changes a request's status. That happens only in the services.

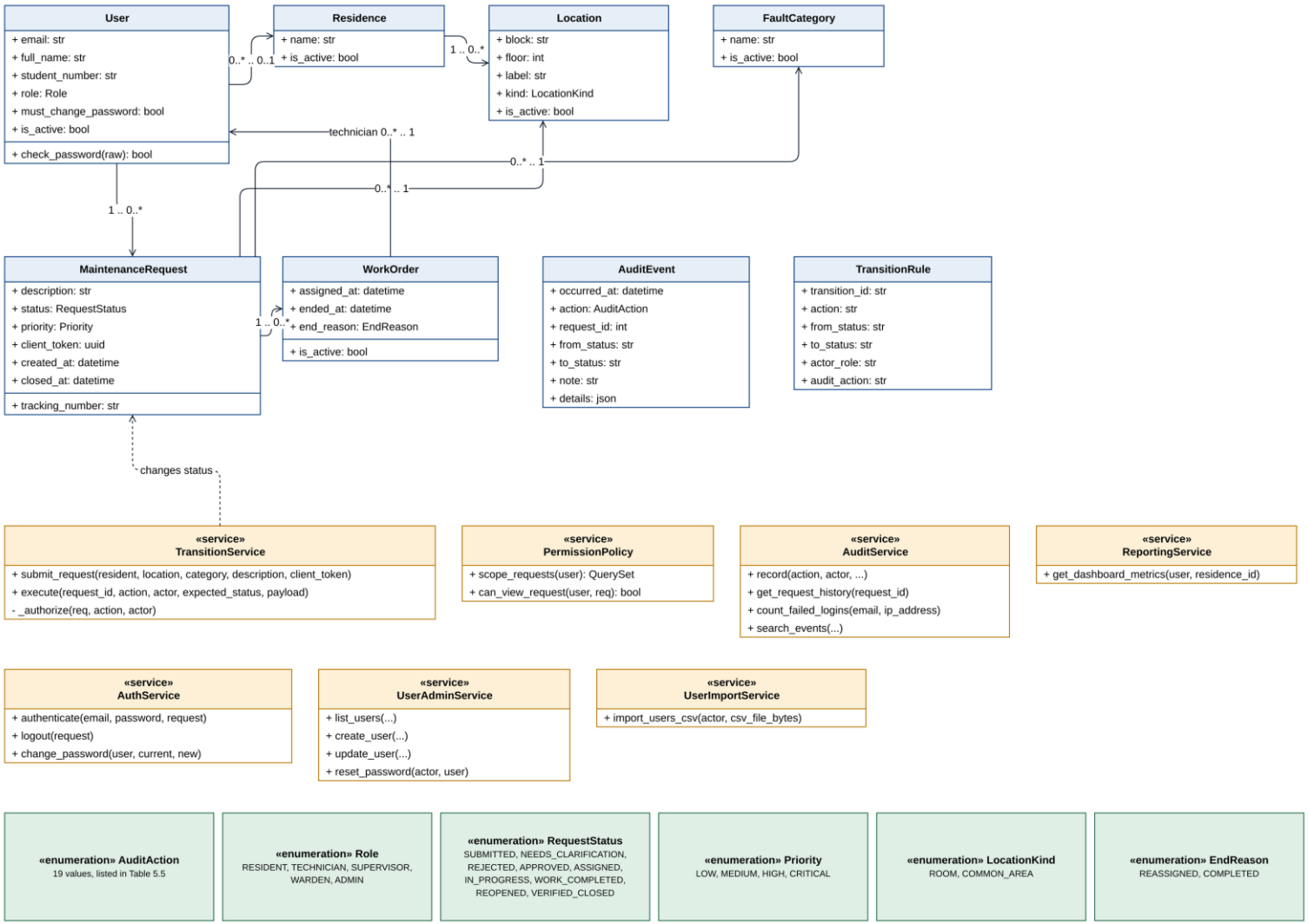


Figure 5.8: Class diagram

Class	Kind	Responsibility
User	Domain	Identity, role, residence (Residents and Wardens), active flag, password hash and the must-change-password flag.
Residence, Location, FaultCategory	Domain	Master data. A Location belongs to one Residence and is a room or a common area.
MaintenanceRequest	Domain	The fault report with its single status, its priority once approved, and its timestamps. It has no method that changes its own status.
WorkOrder	Domain	Assigns a request to a Technician: who, by whom, when, and when and why it ended. No status.
AuditEvent	Domain	One append-only record. It has no update or delete behaviour.
TransitionRule	Domain rule	One row of Table 5.1: action, allowed source states, resulting state, permitted actor and required input.
TransitionService	Service	Runs a transition in one transaction under a row lock, checks the expected status, creates or ends the work order and writes the audit event.
PermissionPolicy	Service	Decides which rows a user may see (scope). Role checks on endpoints use the permission classes in accounts, and TransitionService authorises each action.
AuditService	Service	Writes events inside the caller's transaction, and reads history and search.
AuthService	Service	Sign-in, throttling, sign-out, password change and the forced change.
UserAdminService, UserImportService	Service	User lifecycle with its guards, and all-or-nothing CSV validation and creation.
ReportingService	Service	Dashboard totals scoped to the user's role.
Role, RequestStatus, Priority, LocationKind, EndReason, AuditAction	Enumeration	Values are in Table 5.5.

5.5 User Interface and Interaction Design

The interface is designed for a 360 px phone screen first. It is styled as a maintenance job card: warm paper, navy ink and ochre, ruled tables, request tickets with a tear line, stamped statuses and a track showing how far a request has come. Fonts ship with the application, so nothing loads from outside it. Each role lands on its own home screen. Status always appears as text as well as colour. Actions that cannot be undone (reject, reassign, verify and close) ask for confirmation. The client validates input only to be helpful. The server decides, and its messages are shown beside the field. Every screen has a defined loading, empty, error, conflict and access-denied state, so the interface is designed for things going wrong as well as right.

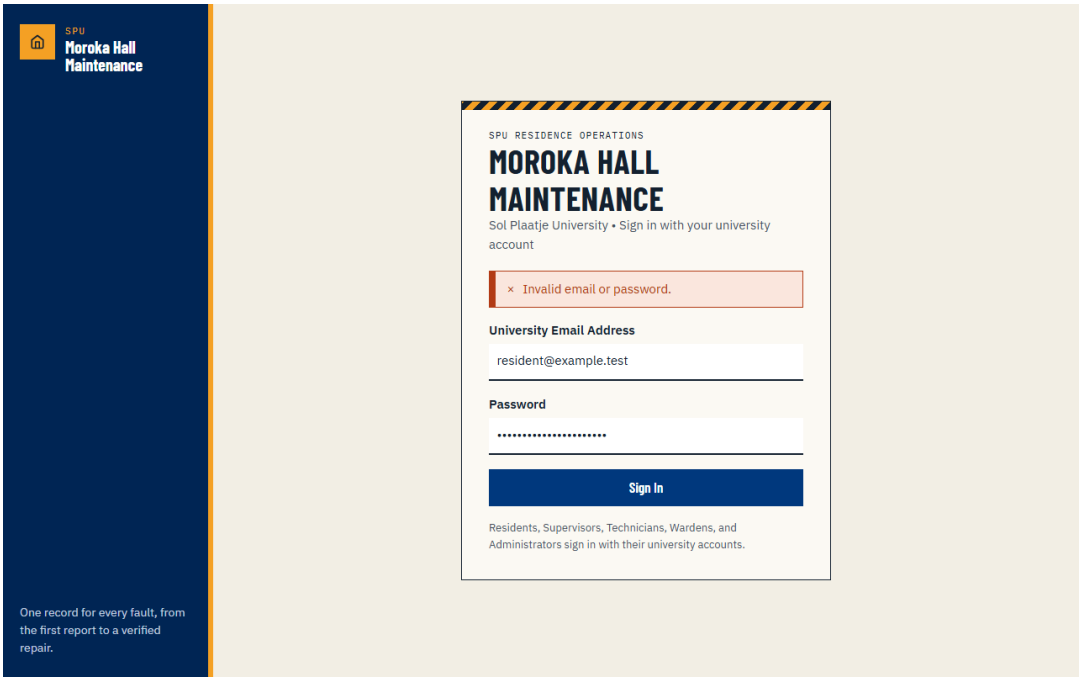
Table 5.3: Screen inventory. Each screen supports an approved requirement.

Screen	Role	Purpose and key elements	Empty, error and denied states	Req.
SCR-01 Sign in	All	Email and password.	Generic error; throttled message; session-expired notice.	FR-01
SCR-02 Change password	All	Forced at first sign-in; voluntary change.	Policy errors beside field.	FR-02
SCR-03 My requests	Resident, Warden	List with status; “Action required” section.	Empty: prompt to report a fault. Load error with retry.	FR-06
SCR-04 Report a fault	Resident, Warden	Location picker, category, description with counter, emergency notice.	Field errors; duplicate submission ignored; load error.	FR-05
SCR-05 Request detail and timeline	All roles, scoped	Status, notes, timeline; The reporter answers a clarification here.	Not found (404) page for out-of-scope request; 409 conflict message.	FR-06, FR-07, FR-18
SCR-06 Triage queue and decision	Supervisor	Queue with filters; approve with priority, reject with reason, ask a question.	Empty queue message; 409 conflict banner with refresh; field errors.	FR-08, FR-09
SCR-07 Dispatch	Supervisor	Approved requests; Technician picker with open-work counts; reassign with reason.	No active Technician message; 409 “already assigned”; field errors.	FR-10, FR-11
SCR-08 Verification review	Supervisor	Completion notes; verify and close, or reopen with feedback.	Empty list; 409 if state changed.	FR-12
SCR-09 My work	Technician	Assigned work by priority; start; complete with required notes.	Empty queue message; notes validation; 409 on double submit.	FR-13 to FR-15
SCR-10 Dashboard	Warden, Supervisor	Counts by status, priority, category; age; mean time to closure.	No-data state; load error; no action controls.	FR-16
SCR-11 Users	Admin	List, create, edit, deactivate, reset password.	Refusal messages for guarded actions.	FR-03
SCR-12 CSV import	Admin	Upload, row-level validation report, one-time credentials download.	Report listing every invalid row; file rejected messages.	FR-04
SCR-13 Audit log	Admin	Filter by actor, action, request, date.	No results; 403 page for other roles.	FR-18
System pages	All	Access denied (403), not found (404), session expired, network error.	Shown wherever the API returns the corresponding error.	NFR-01

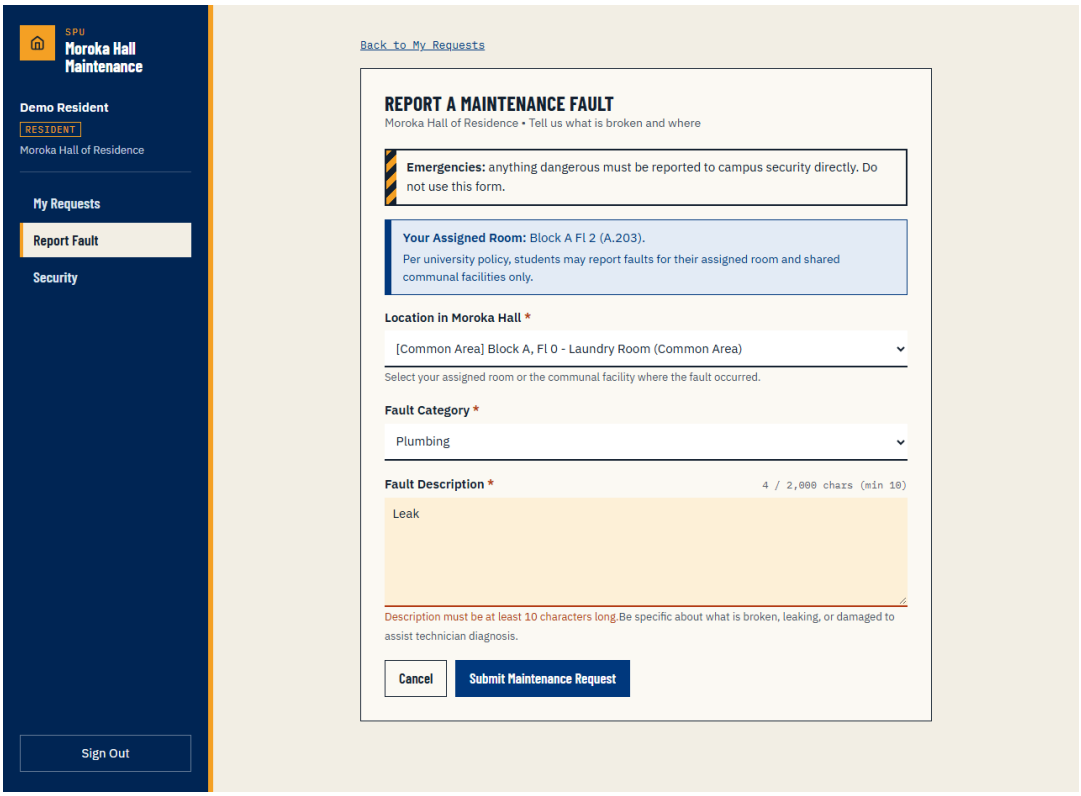
Table 5.4: Screenshots from the running system, covering sign-in, a core transaction, an admin workflow, validation and error states, and finished results.

Shot	User	Purpose and main action	Requirement / workflow
S-01	All	Sign-in screen with a failed attempt showing the generic error.	FR-01; AC-01.2

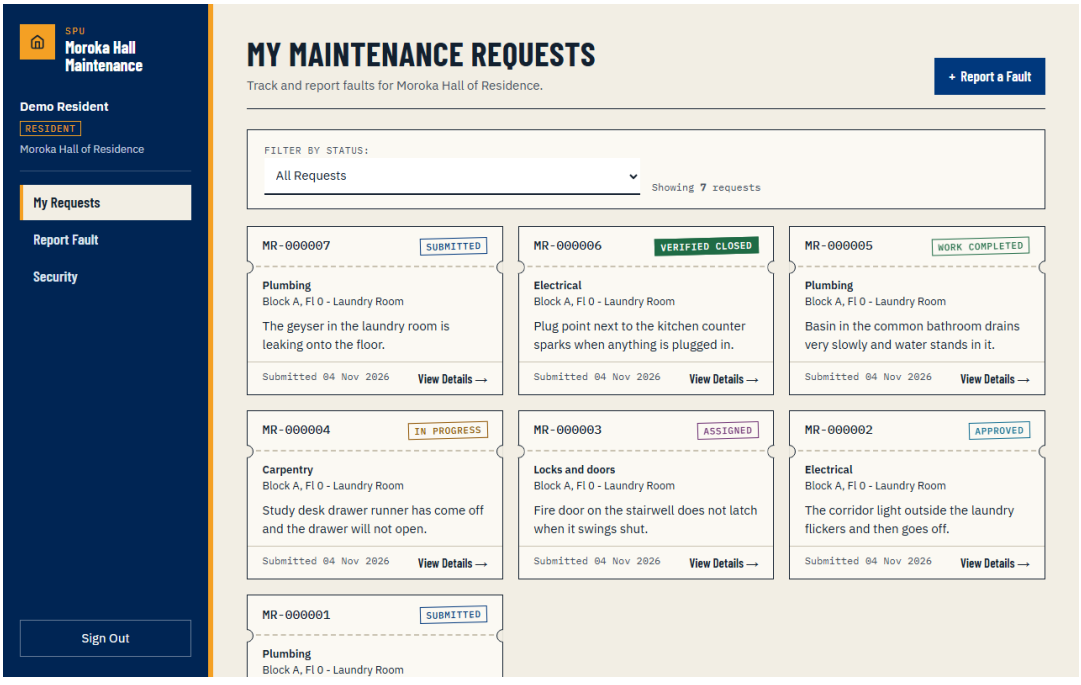
Shot	User	Purpose and main action	Requirement / workflow
S-02	Resident	Report a fault: description too short, field error and emergency notice visible (validation state).	FR-05; AC-05.2
S-03	Resident	My requests after submission: new request with tracking ID and status Submitted (result state).	FR-05, FR-06
S-04	Supervisor	Triage queue and approval with priority.	FR-08, FR-09
S-05	Supervisor	Dispatch: conflict message when another Supervisor has already assigned the request (error state).	FR-10; AC-10.4
S-06	Technician	Complete work: Confirm stays disabled until the notes reach 20 characters (validation state), then accepted.	FR-15; AC-15.2
S-07	Supervisor	Verification review: verify and close.	FR-12; AC-12.1
S-08	Resident	Request detail showing the full timeline to Verified & Closed (completed state).	FR-06
S-09	Warden	Dashboard: read-only, no action controls, no resident identity.	FR-16; AC-16.1
S-10	Admin	CSV import validation report with invalid rows and no accounts created (administrative workflow, error state).	FR-04; AC-04.2
S-11	Any	Access-denied page (permission-denied state).	NFR-01



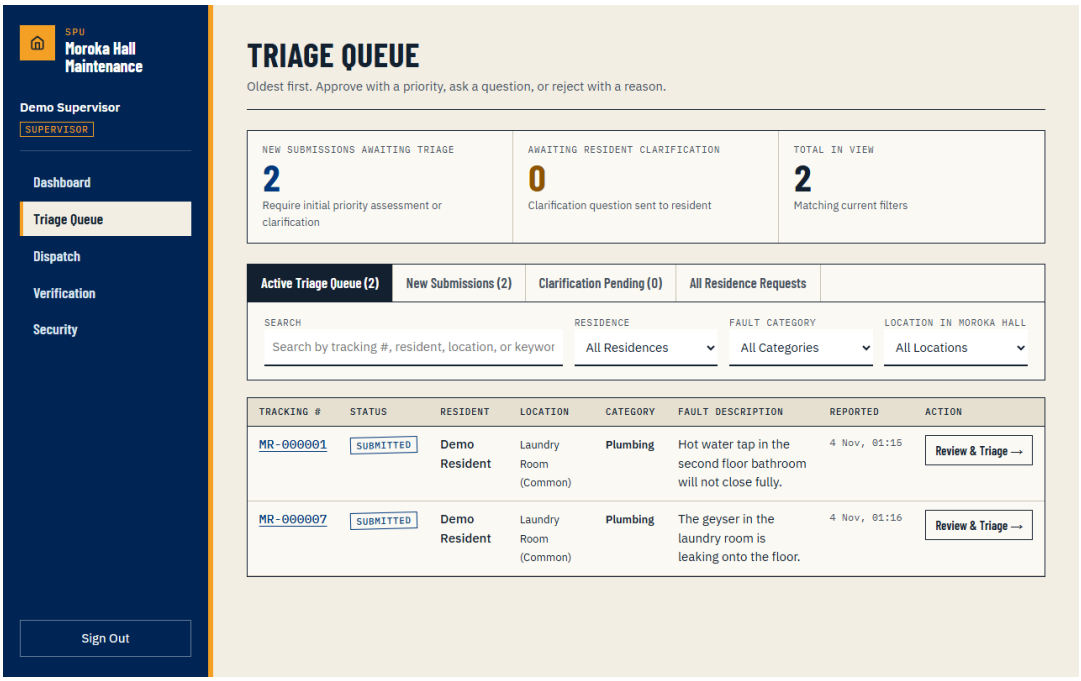
S-01. Any user. Sign-in refused with the one generic message (FR-01, AC-01.2).



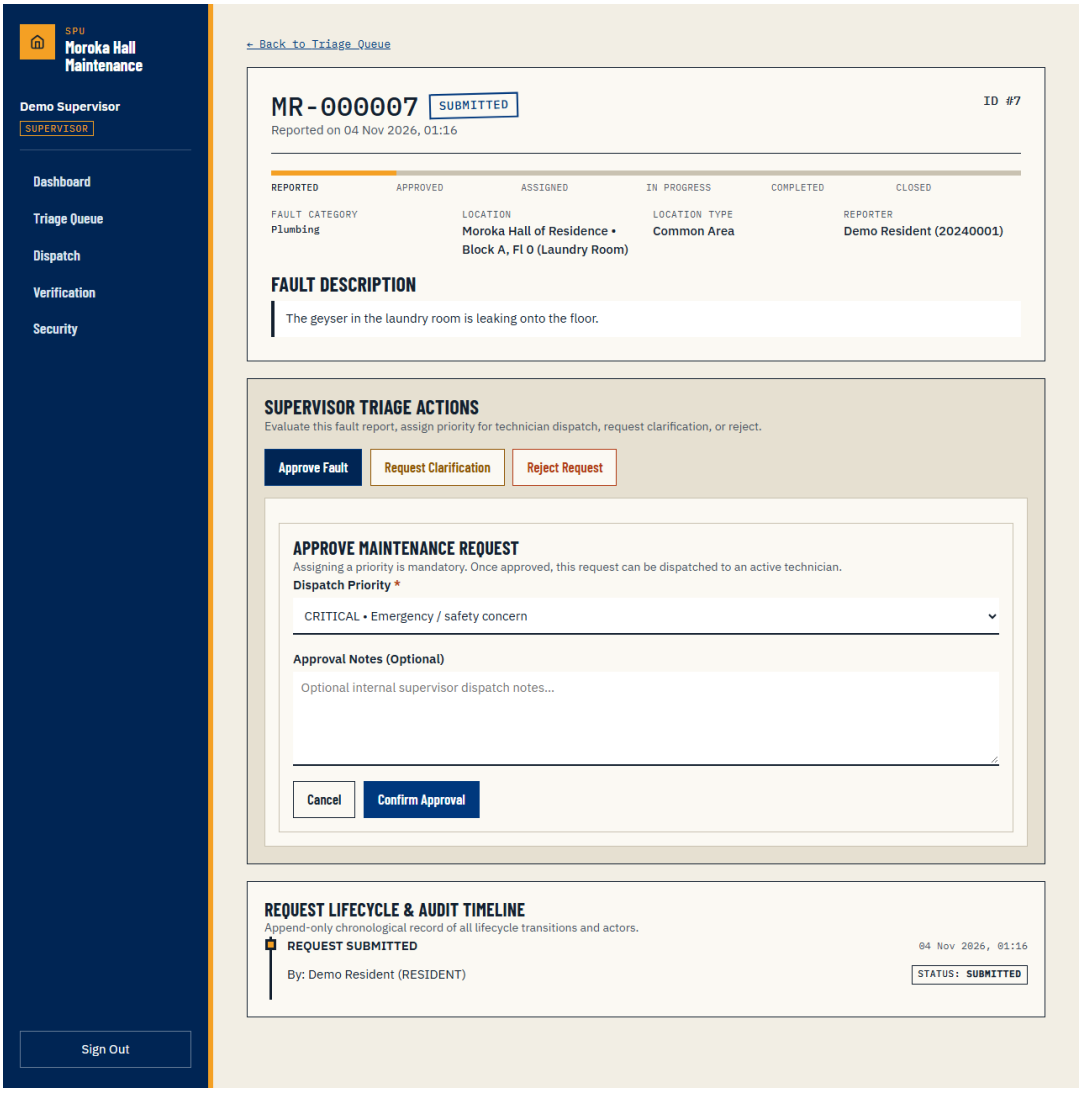
S-02. Resident. A description that is too short shows a field error, with the emergency notice always visible (FR-05, AC-05.2, AC-05.4).



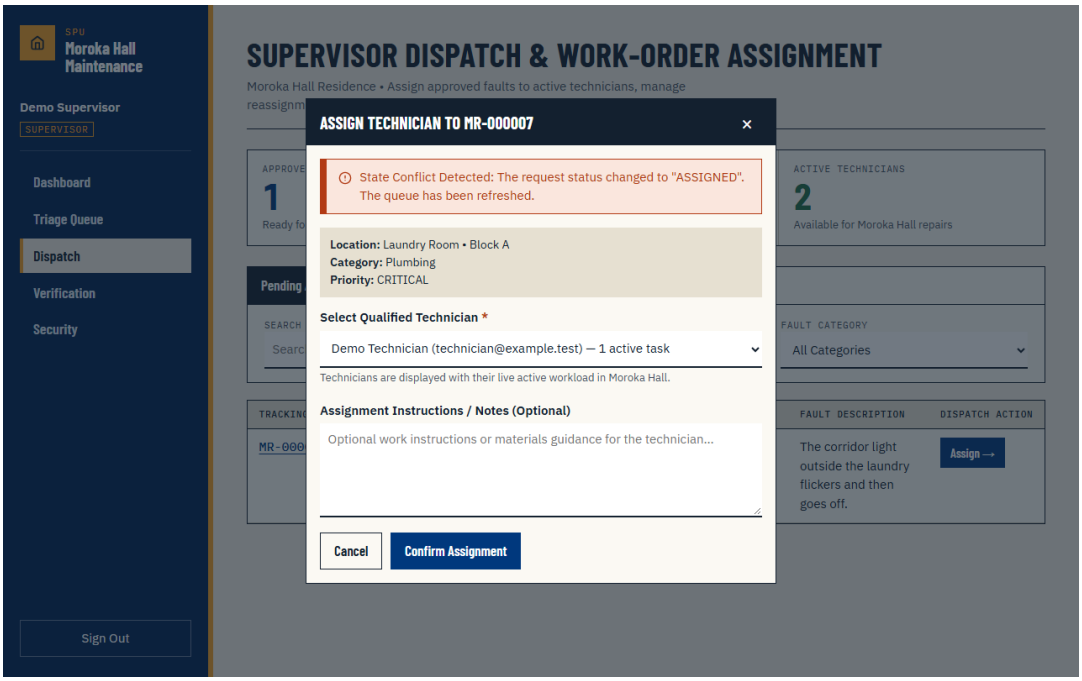
S-03. Resident. My Requests after submitting, with the tracking ID and status on each ticket (FR-05, FR-06).



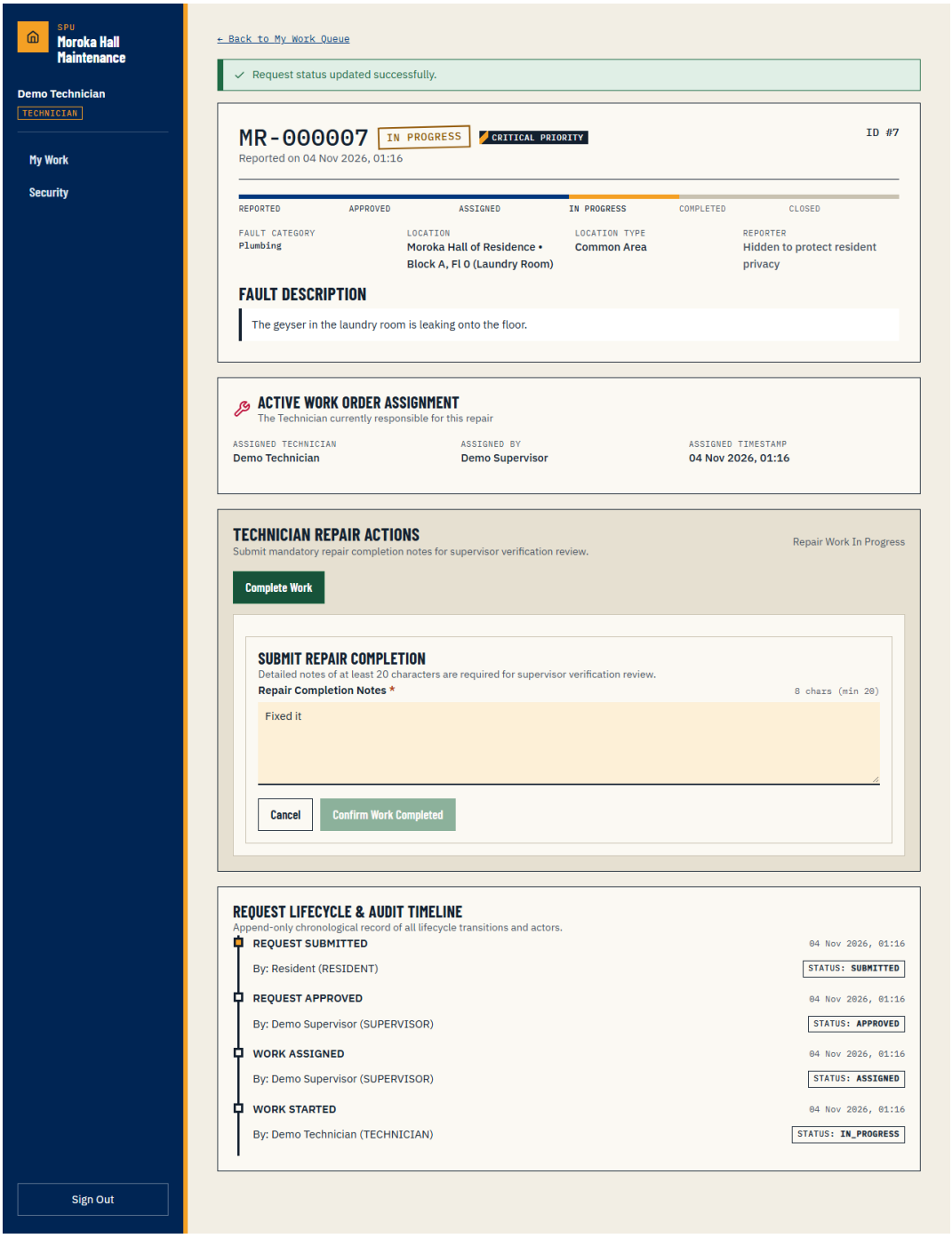
S-04a. Supervisor. Triage queue, oldest first, with residence and category filters (FR-08).



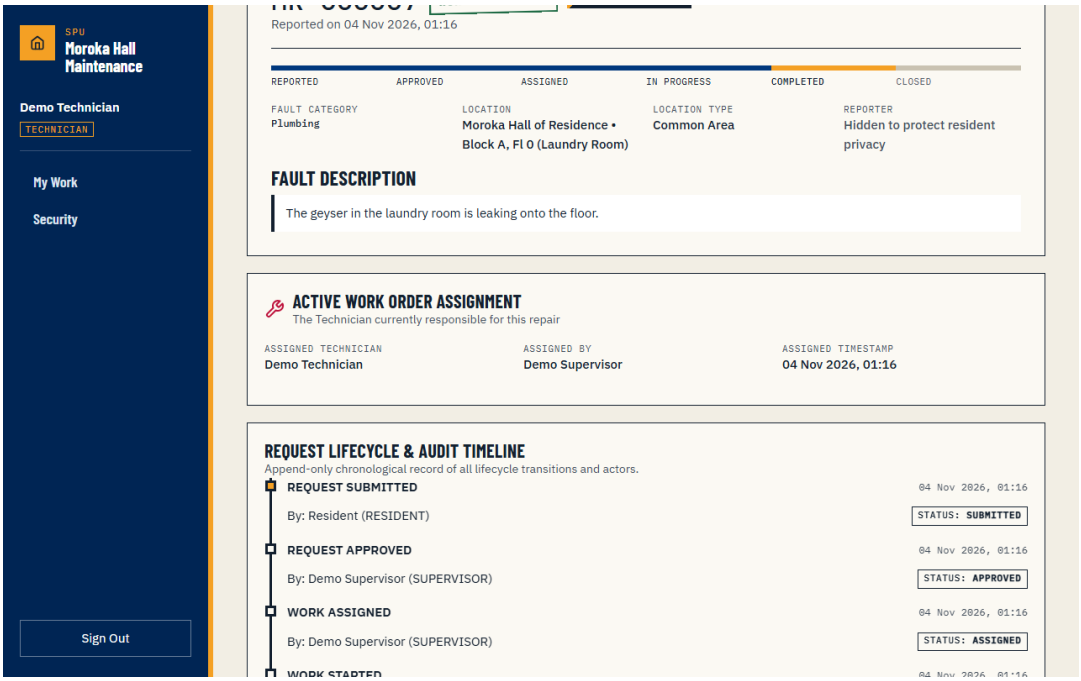
S-04b. Supervisor. Approving a request sets its priority (FR-09).



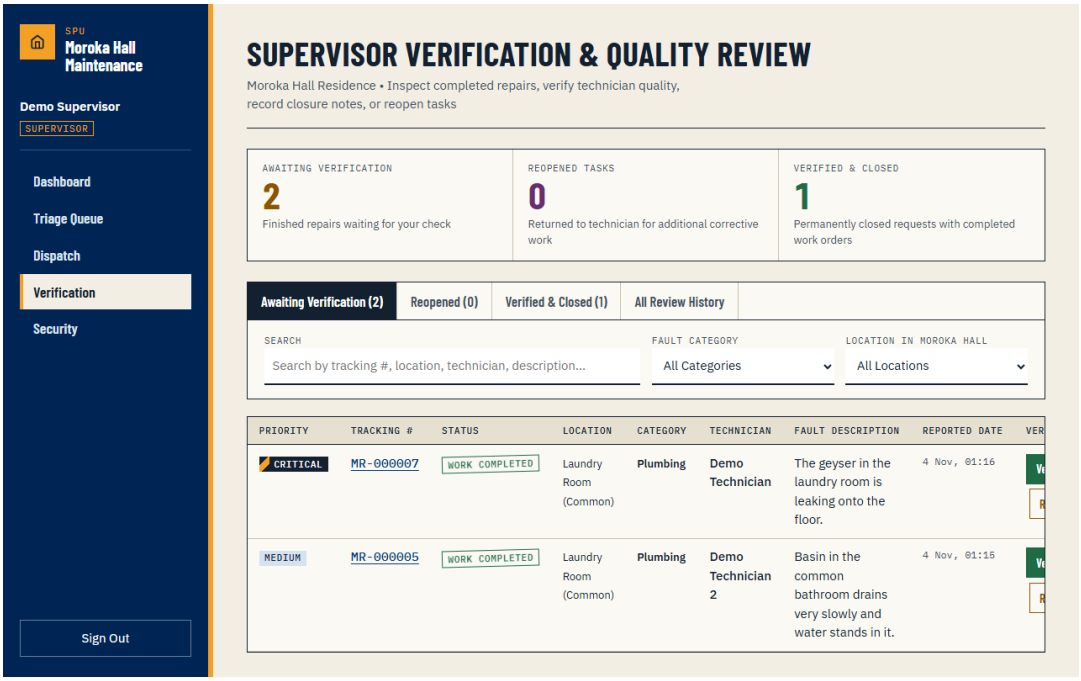
S-05. Supervisor. Another Supervisor assigned the request first, so this attempt gets a conflict message (FR-10, AC-10.4).



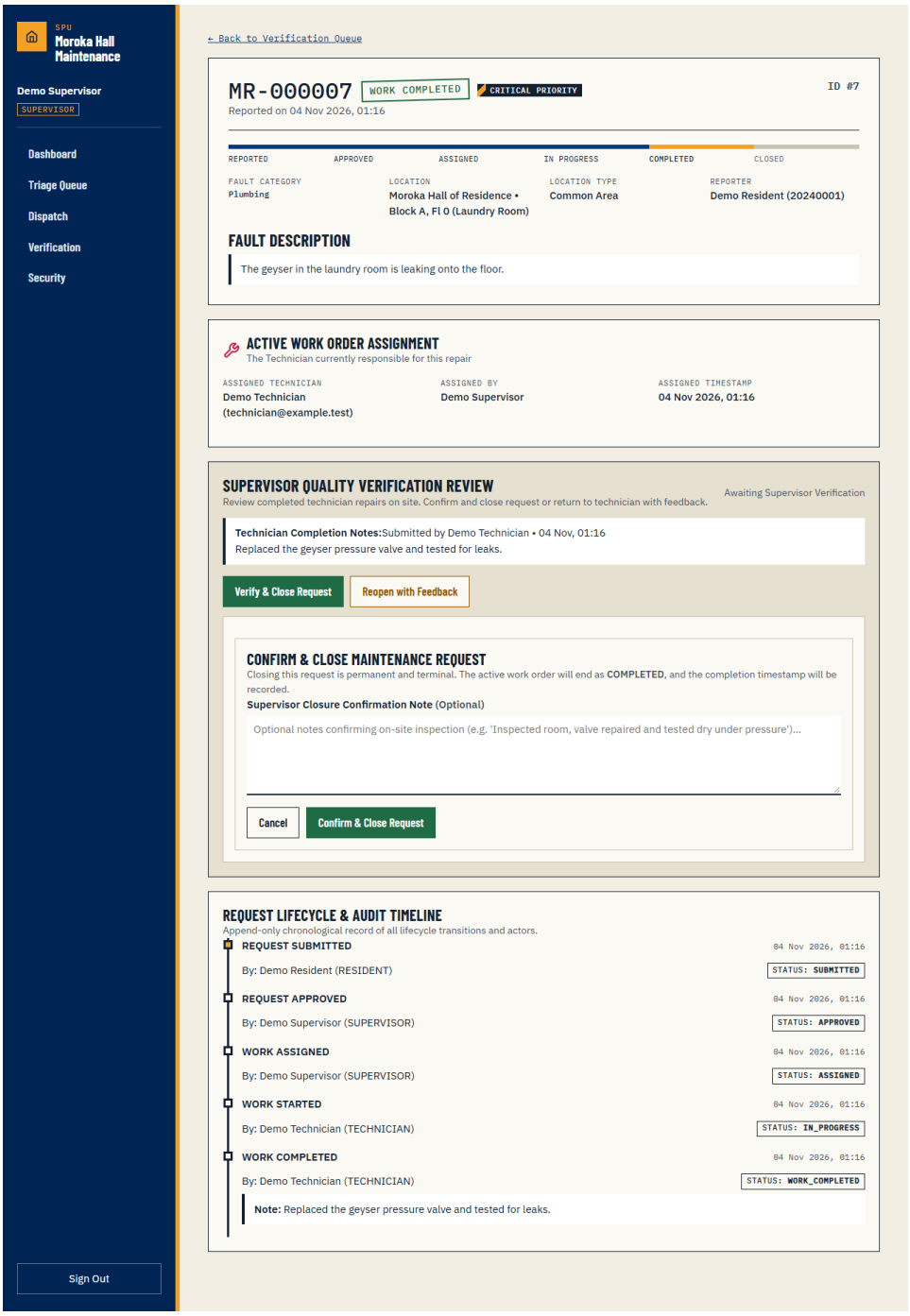
S-06a. Technician. Confirm stays disabled while notes are under 20 characters (FR-15, AC-15.2).



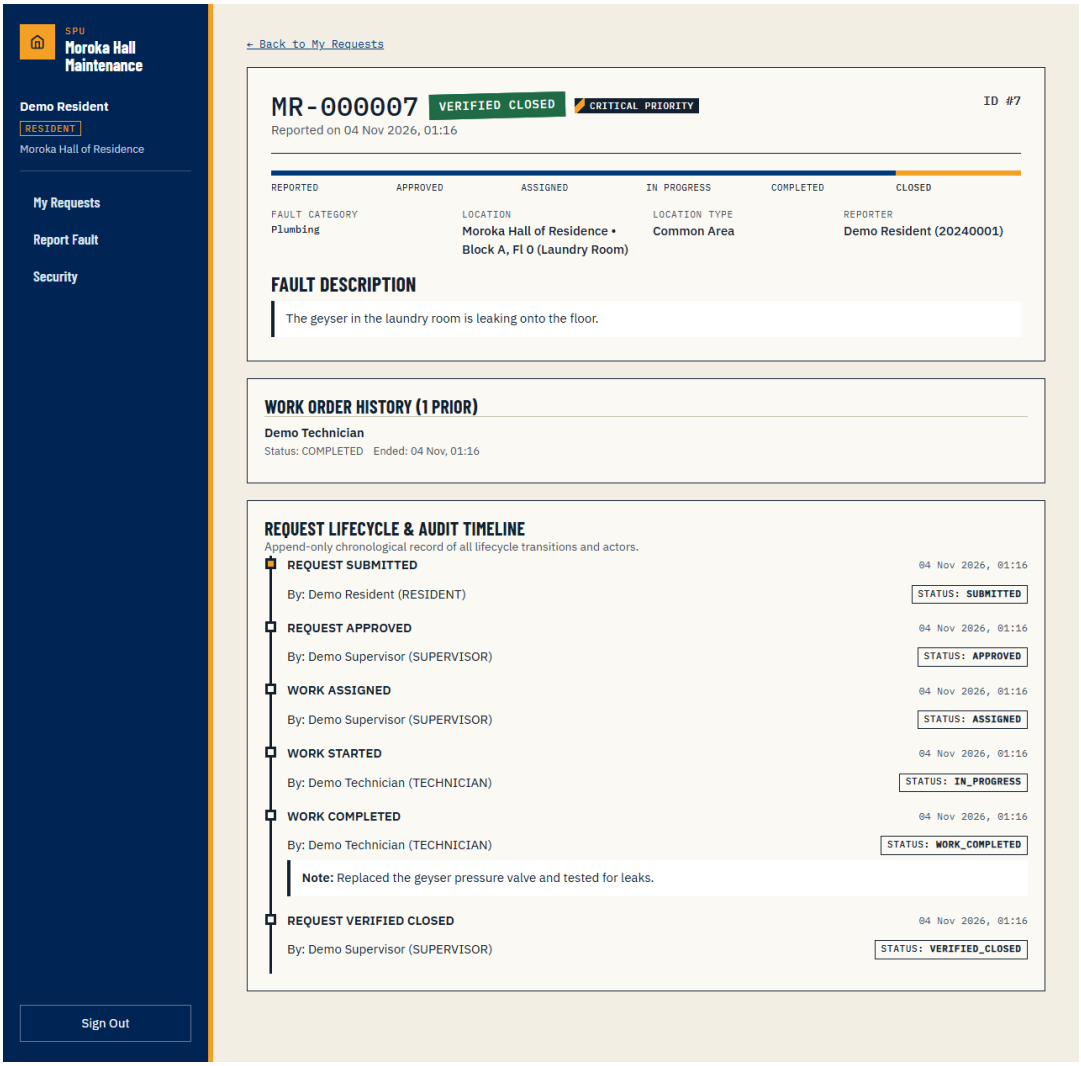
S-06b. Technician. Valid notes move the request to Work Completed (FR-15, AC-15.1).



S-07a. Supervisor. Finished repairs waiting for review (FR-12).



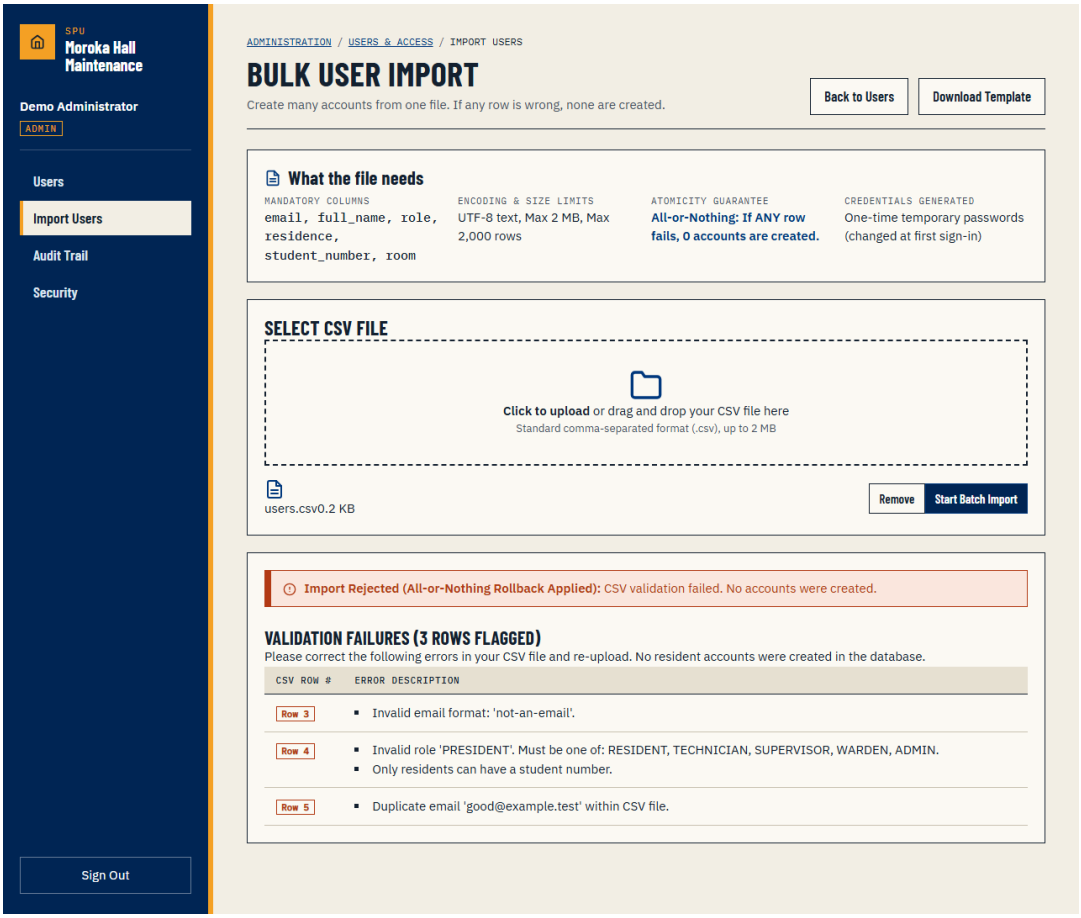
S-07b. Supervisor. Verify and close, with the completion notes shown (FR-12, AC-12.1).



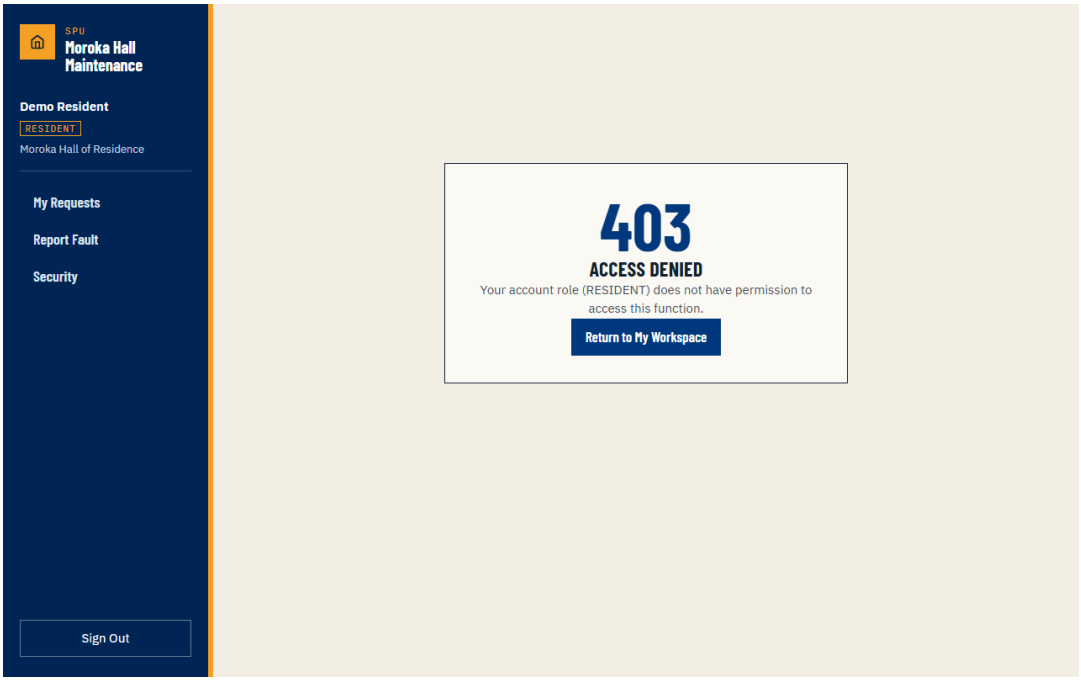
S-08. Resident. The full timeline of a Verified Closed request (FR-06).



S-09. Warden. Read-only dashboard with no action controls and no resident identity (FR-16, AC-16.1).



S-10. Admin. CSV validation report: every bad row listed and no accounts created (FR-04, AC-04.2).



S-11. Resident. Access-denied page for a function of another role (NFR-01).

5.6 Data Design

PostgreSQL 16 holds everything: users, master data, requests, work orders and the audit log. It also enforces the integrity rules itself. Primary keys are 64-bit identity columns, timestamps are stored in UTC and shown in South African Standard Time, and nothing is hard-deleted. Every foreign key is RESTRICT and records are deactivated instead. The tracking ID (for example MR-000123) is built from the request's primary key.

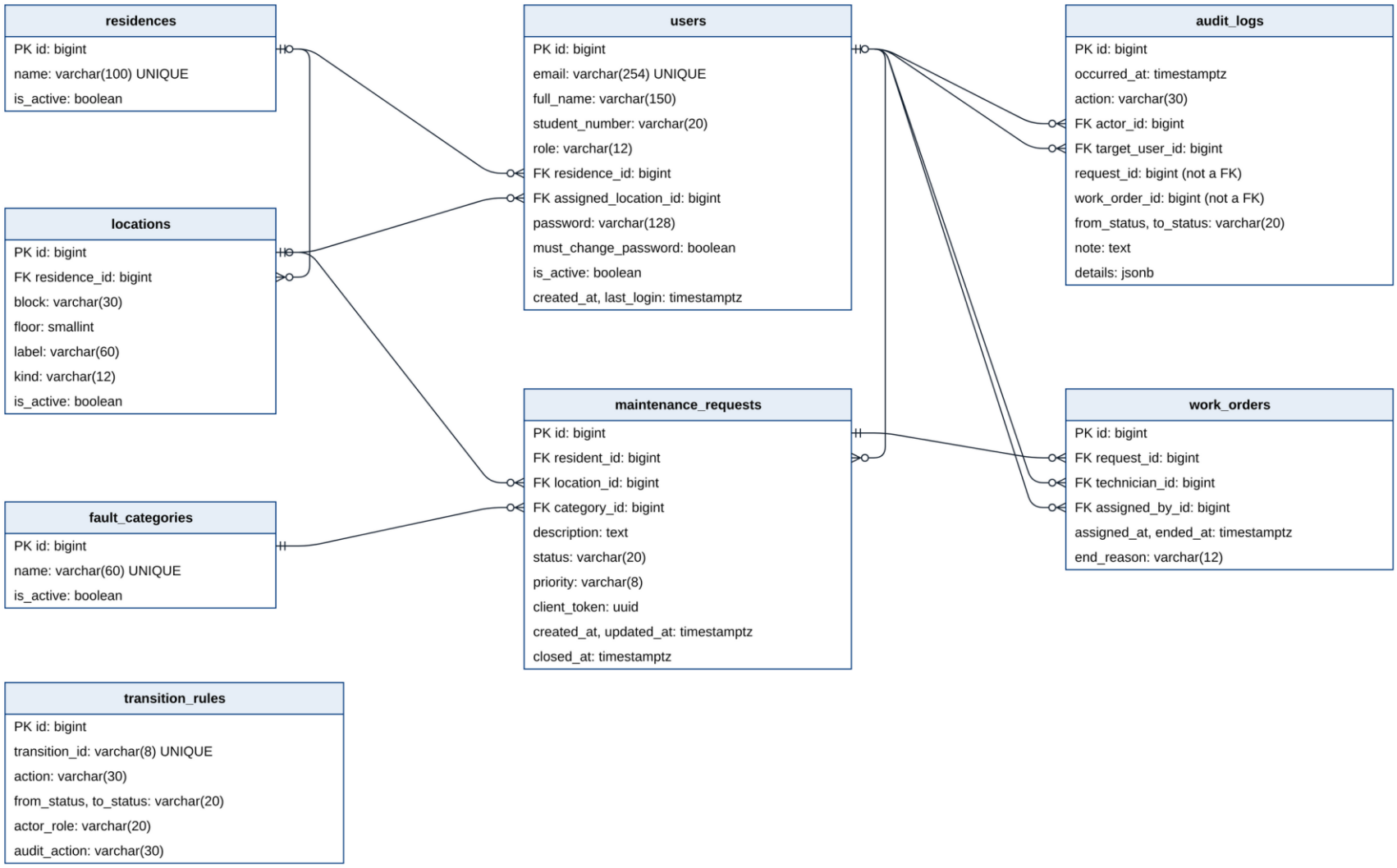


Figure 5.9: Entity-relationship diagram

Table 5.5: Enumerations

Enumeration	Values
Role	RESIDENT, TECHNICIAN, SUPERVISOR, WARDEN, ADMIN
RequestStatus	SUBMITTED, NEEDS_CLARIFICATION, REJECTED, APPROVED, ASSIGNED, IN_PROGRESS, WORK_COMPLETED, REOPENED, VERIFIED_CLOSED
Priority	LOW, MEDIUM, HIGH, CRITICAL
LocationKind	ROOM, COMMON_AREA
EndReason	REASSIGNED, COMPLETED
AuditAction	REQUEST_SUBMITTED, CLARIFICATION_REQUESTED, CLARIFICATION_PROVIDED, REQUEST_APPROVED, REQUEST_REJECTED, WORK_ASSIGNED, WORK_REASSIGNED, WORK_STARTED, WORK_COMPLETED, REQUEST_VERIFIED_CLOSED, REQUEST_REOPENED, USER_CREATED, USER_IMPORTED, USER_ROLE_CHANGED, USER_DEACTIVATED, USER_REACTIVATED, PASSWORD_RESET, PASSWORD_CHANGED, LOGIN_FAILED

Table 5.6: Relationships, cardinality and deletion rules. Every foreign key is ON DELETE RESTRICT.

Parent	Child	Cardinality	Note
residences	users	1 to 0..*	Residence is set exactly for Residents and Wardens.
residences	locations	1 to 0..*	
locations	users	0..1 to 0..*	Assigned room of a Resident.
users (reporter)	maintenance_requests	1 to 0..*	A Resident, or a Warden reporting for a Resident.
locations	maintenance_requests	1 to 0..*	Must be in the reporter's residence (INV-4).
fault_categories	maintenance_requests	1 to 0..*	
maintenance_requests	work_orders	1 to 0..*	Over time many; at most one active (partial unique index).
users (Technician)	work_orders	1 to 0..*	Assigned Technician.
users (Supervisor)	work_orders	1 to 0..*	Assigned by.
users	audit_logs	0..1 to 0..*	Actor; null when no user is signed in. A second optional link records the target user.
maintenance_requests, work_orders	audit_logs	0..1 to 0..*	Plain bigint columns, not foreign keys: audit rows stay independent of the records they describe.

Table 5.7: Data dictionary

Table	Column	Type	Constraints and notes
residences	id; name; is_active	bigint; varchar(100); boolean	PK; name UNIQUE NOT NULL.
users	id	bigint	PK.
	email	varchar(254)	NOT NULL; UNIQUE on lower(email).
	full_name	varchar(150)	NOT NULL.
	student_number	varchar(20)	NULL; UNIQUE where not null; required when role is RESIDENT.
	role	varchar(12)	NOT NULL; CHECK in Role.
	residence_id	bigint	FK residences. CHECK: NOT NULL exactly when role is RESIDENT or WARDEN.
	assigned_location_id	bigint	FK locations, nullable. CHECK: set only for a Resident.
	password; must_change_password	varchar(128); boolean	Argon2id hash; true after Admin issues a temporary password.
	is_active; created_at; last_login	boolean; timestamptz; timestamptz	NOT NULL except last_login.
locations	id; residence_id	bigint	PK; FK residences.
	block; floor; label; kind; is_active	varchar(30); smallint; varchar(60); varchar(12); boolean	UNIQUE (residence_id, block, floor, label); CHECK kind in LocationKind; ground floor is 0.
fault_categories	id; name; is_active	bigint; varchar(60); boolean	PK; name UNIQUE NOT NULL.
maintenance_requests	id	bigint	PK.
	resident_id; location_id; category_id	bigint	NOT NULL; FKs to users, locations, fault_categories.
	description	text	NOT NULL; CHECK length of trimmed text between 10 and 2,000.
	status	varchar(20)	NOT NULL; CHECK in RequestStatus.
	priority	varchar(8)	CHECK in Priority; CHECK NOT NULL for APPROVED, ASSIGNED, IN_PROGRESS, WORK_COMPLETED, REOPENED, VERIFIED_CLOSED.
	client_token	uuid	NOT NULL; UNIQUE (resident_id, client_token) gives idempotent submission.
	created_at; updated_at; closed_at	timestamptz	CHECK: closed_at NOT NULL exactly when status is REJECTED or VERIFIED_CLOSED.

Table	Column	Type	Constraints and notes
	Indexes		(status, created_at) for the triage queue; (resident_id, created_at DESC); (location_id); (category_id).
work_orders	id; request_id; technician_id; assigned_by_id	bigint	PK; FKs; all NOT NULL.
	assigned_at; ended_at; end_reason	timestamptz; timestamptz; varchar(12)	CHECK end_reason in EndReason; CHECK end_reason IS NULL exactly when ended_at IS NULL.
	Partial unique index		UNIQUE (request_id) WHERE ended_at IS NULL: at most one active work order per request (BR-01). Index (technician_id) WHERE ended_at IS NULL for queues.
audit_logs	id; occurred_at; action	bigint; timestamptz; varchar(30)	PK; NOT NULL; CHECK action in AuditAction.
	actor_id; request_id; work_order_id; target_user_id	bigint	Nullable. actor_id and target_user_id are FKs; request_id and work_order_id are plain bigints (Table 5.6). Set according to the event.
	from_status; to_status; note; details	varchar(20); varchar(20); text; jsonb	Nullable except details (default empty object). Notes carry reasons, questions, feedback and completion notes; details carry, for example, the attempted email and IP address of a failed sign-in or both Technicians on a reassignment.
	Protection and indexes		BEFORE UPDATE OR DELETE row trigger and BEFORE TRUNCATE statement trigger raise an exception; the application role has INSERT and SELECT only. The action CHECK is NOT VALID, so it binds new rows without rewriting audit history (CH-12). Indexes (request_id, occurred_at), (actor_id, occurred_at), (action, occurred_at).
transition_rules	id; transition_id; action; from_status; to_status; actor_role; audit_action; description	bigint; varchar	PK; transition_id UNIQUE; CHECK statuses in RequestStatus. One row per legal source state of Table 5.1, synced by the seed command.

Normalisation and constraints. The schema is in third normal form: each fact is stored once and every other column depends on the key. I made one exception. `closed_at` could be worked out from the status and the audit log, but I store it to keep reporting queries simple, and a CHECK constraint keeps it consistent with the status. The indexes support the three busiest queries (triage queue, Resident list, Technician queue) and audit lookups.

Table 5.8: Critical transactions and their integrity rules. Each is one database transaction. If any step fails, none of its changes remain.

Transaction	Steps (atomic unit)	Integrity rules	Concurrency and failure behaviour
CT-01 Assign work order (FR-10)	Lock request row; check status equals expected and is APPROVED; lock Technician row and check active and role; insert work order; set request ASSIGNED; insert audit event.	BR-01; INV-1, INV-3; partial unique index.	A second Supervisor waits, then receives 409. A duplicate insert violates the index and is mapped to 409. Any failure rolls back all steps (TC-23 to TC-25).
CT-02 Reassign work order (FR-11)	Lock request; check state and reason; lock new Technician; end old work order with REASSIGNED; insert new work order; set ASSIGNED; insert audit event naming both Technicians.	BR-01, BR-02, BR-03; INV-1, INV-3.	Ending and creating happen together, so the request is never unassigned or double assigned (TC-26, TC-27).
CT-03 Complete work (FR-15)	Lock request; check IN_PROGRESS and assigned Technician; validate trimmed notes; set WORK_COMPLETED; store notes in the audit event.	BR-02; INV-2.	A double submit receives 409 on the second call; invalid notes change nothing (TC-30, TC-31).
CT-04 Bulk import (FR-04)	Validate the whole file in memory; create all users and their audit events; or create none.	BR-07; unique email.	Any invalid row aborts the import with a row-level report (TC-07, TC-08).

Data invariants. These hold after every operation and are checked by an invariant checker that runs after the randomised tests (NT-03).

- **INV-1:** An active work order exists for a request exactly when its status is ASSIGNED, IN_PROGRESS, WORK_COMPLETED or REOPENED, and never more than one.
- **INV-2:** For every request, the new state of its latest status-changing audit event equals its current status.
- **INV-3:** The Technician on every active work order is an active user with role TECHNICIAN.
- **INV-4:** The Location of a request is in the same residence as its reporter.
- **INV-5:** Priority is set for every request that has been approved, and closed_at is set exactly for terminal states.
- **INV-6:** No audit event is ever changed or removed.

INV-4 spans two tables, so it cannot be a simple CHECK constraint. It is enforced by the service layer and verified by the invariant checker and tests.

5.7 Security, Privacy and Access Control

Authentication. A user signs in with email and password. The server checks the Argon2id hash, issues a new session identifier and sets an HttpOnly, SameSite=Lax cookie. The browser sends that cookie, plus a CSRF token header, on every call that changes data. On each request the server loads the user and role from the database, so deactivation or a role change takes effect at once. A user on a temporary password can use nothing but sign-out and the password change until they set their own; this is enforced by middleware on the server. Failed sign-ins go into the audit log, and the throttle counts them per account and per IP address over one minute.

Role and permission matrix

Function	Resident	Technician	Supervisor	Warden	Admin	Scope rule
Report a fault	✓	—	—	✓ For a Resident	—	Own residence locations
View requests	Own	Assigned only	All	Own residence; no resident identity	All (read only)	Role-scoped queries
Answer a clarification	Own report	—	—	Own report	—	Owner only
Triage; assign; reassign	—	—	✓	—	—	—
Start and complete work	—	Assigned only	—	—	—	Active work order
Verify and close; reopen	—	—	✓	—	—	—
Dashboard	—	—	All residences	Own residence	—	Read-only
Request history	Own	Assigned only	✓	Own residence; no resident identity	✓	—
Search audit log	—	—	—	—	✓	—
Manage users; CSV import	—	—	—	—	✓	—
Change own password	✓	✓	✓	✓	✓	Needs current password

The server enforces this through PermissionPolicy, the permission classes and TransitionService. Rows outside a user's scope are simply invisible, so an ID outside the scope gives 404 and does not reveal that the record exists. The Admin cannot make maintenance decisions, and only a Supervisor can approve, assign or verify (BR-10).

Data classification matrix

Data	Class	Why collected	Storage	Shown to	Retention and deletion
Password	Secret	Authentication	Argon2id hash only; never logged or returned	Nobody	Replaced on change; never deleted by the system
Temporary password	Secret	First sign-in	Hash only; shown once to the Admin in the import file or reset response	Admin once	Invalidated at first change
Email, full name	Personal	Identify and sign in	Plain, in users	Own user; Admin; Supervisor (name on requests). Technician email: Supervisor and Admin only	Deactivate; no hard delete (LIM-06)
Student number, residence	Personal	Match a Resident to a residence	Plain, in users	Own user; Admin	As above
Fault description, notes	Operational (may contain personal details)	Describe and resolve the fault	In requests and audit_logs	Resident own; Supervisor; Technician (assigned); Warden (own residence, no identity)	Kept for history; retention is an institutional decision
Audit events	Operational	Accountability	Append-only table	Owner, Supervisor, Admin, assigned Technician and Warden (request timeline, no resident identity for the last two); Admin (search)	Never altered or deleted by the application
Not collected	—	Phone numbers, ID numbers, photographs, GPS: no requirement needs them	—	—	—

Security controls

Threat	Control	Verified by
Credential stuffing and brute force	Argon2id; throttle of 5 failures per minute per account and per IP; one generic failure message; failures audited; X-Forwarded-For trusted only from the proxy.	TC-02, TC-03, NT-02
Session theft, fixation, CSRF	HttpOnly, SameSite=Lax cookies (marked Secure outside debug mode); session rotated at sign-in; 8-hour expiry; CSRF token on writes; strict Content-Security-Policy.	NT-02, NT-09, ST-05
Parameter tampering and mass assignment	`status`, `role`, priority and ownership fields are read-only in serialisers; every status change goes through the transition service.	ST-03, TC-35

Threat	Control	Verified by
Broken object-level authorisation	Role-scoped querysets; out-of-scope objects return 404; matrix test covers every endpoint.	NT-01, ST-04, TC-14
Privilege escalation	Role read from the database on every request; last-Admin protection; Admin holds no maintenance rights.	NT-01, TC-06
Audit tampering	Trigger rejecting UPDATE, DELETE and TRUNCATE; application role lacks those privileges; events written in the same transaction as the change.	TC-39
Injection (SQL, script, CSV)	ORM parameterisation with no string-built SQL; React output escaping with no raw HTML injection; strict CSV parsing with size and row limits.	ST-01, ST-02, TC-09
Personal-information exposure	Wardens and Technicians never receive resident identity; minimal fields collected; debug output disabled.	ST-06, TC-28, TC-36
Credential provisioning	Temporary passwords are random, stored only as hashes, shown once, and force a change at first sign-in; issue and reset are audited.	TC-05, TC-07, LIM-02
Denial of service	Request-body limit of 2 MB; pagination capped at 50; database statement timeout.	TC-09, TC-16
Secrets and supply chain	Secrets in environment variables; untracked `.env` with a documented `.env.example`; lock files; dependency audits in CI.	NT-09, NT-07
Transport exposure	The assessed system runs over HTTP on localhost, and the database is on an internal network only. TLS is out of scope for this evaluation.	LIM-10

Privacy. The system holds personal information: names, email addresses, student numbers, residence and the text of fault reports. I followed the ideas in the Protection of Personal Information Act 4 of 2013: use the data only for maintenance, collect only what is needed, restrict access by role and apply technical safeguards (Republic of South Africa, 2013). Retention and data-subject requests are for the institution to decide and the system does not automate them (LIM-06). I am not claiming legal compliance.

The security test cases are in Section 7.3 (NT-01 to NT-09 and ST-01 to ST-06) and are traced in the table above.

5.8 Deployment and Runtime Design

The assessed system runs on one machine in Docker containers (Docker Compose). It is not on a public server or the cloud (LIM-10). The browser is the only outside client. It talks to the nginx proxy, which serves the built client and forwards `/api` to the API container. The database sits on an internal network and publishes no port to the host.

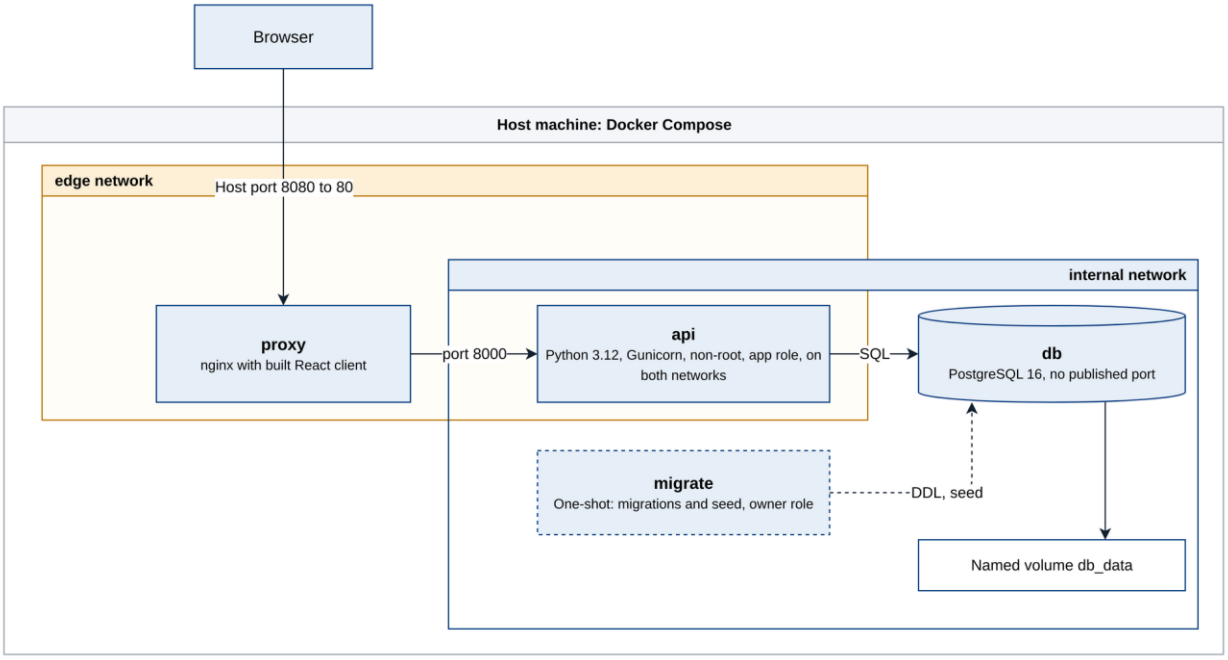


Figure 5.10: Deployment view

Service	Base	Exposure	Notes
proxy	nginx	Host port 8080 to container port 80	Serves the built React client; proxies `/api`; sets security headers; limits request bodies to 2 MB.
api	Python 3.12 with Gunicorn	Edge and internal networks; no published port	Runs as a non-root user with the restricted database role; health endpoint `/api/health/`.
db	PostgreSQL 16	Internal network only; no published port	Named volume; health check; initialisation creates the owner and runtime roles.
migrate	Same image as api	One-shot	Runs migrations as the owner role and applies idempotent seed data.

- **Networks.** An edge network joins proxy and api; an internal network joins api, db and migrate.
- **Configuration.** Environment variables from an untracked `.env` file; `.env.example` documents every variable.
- **Profiles.**The evaluation profile runs over HTTP on localhost and seeds demo accounts and sample requests. No TLS profile is included (LIM-10).

6. SYSTEM IMPLEMENTATION

6.1 Codebase and Repository Structure

Everything is in one Git repository, organised by architecture component. The complete source is on GitHub (Section 11.1) and this report does not reproduce code.

Path	Contents	Architecture component
backend/config/	Django settings, URL routing, environment handling, forced-password middleware, error format, pagination	API layer
backend/apps/accounts/	User model, AuthService, UserAdminService, UserImportService, views	Service and API layers
backend/apps/locations/	Residence, Location, FaultCategory models and the seed command	Data layer
backend/apps/workflow/	MaintenanceRequest, WorkOrder, TransitionRule table, TransitionService, PermissionPolicy, ReportingService, request, work-queue and dashboard views, sample-data commands	Service, API and data layers
backend/apps/audit/	AuditEvent model, AuditService, trigger and privilege migrations, audit views	Service and data layers
backend/data/seed/	Room and common-area list loaded by the seed command	Data layer
backend/tests/	pytest suites: API, concurrency, database, invariant checker, generated matrices, repository rules, regressions	Verification
frontend/src/pages/, components/	One page per screen, shared components, styles in index.css	Web client
frontend/src/api/	API client; schema.d.ts generated from the OpenAPI schema	Web client
frontend/e2e/, loadtests/	Playwright journeys; Locust profile	Verification
deploy/	Database initialisation script (roles, privileges) and security header check	Database; verification
frontend/nginx/	nginx configuration: security headers, body limit, /api proxy	Reverse proxy
docs/	OpenAPI schema, traceability, deviations, draw.io diagrams, test evidence	Verification
.github/workflows/	CI: lint, type check, tests, audits, clean stack and browser journeys	Verification

Principal dependency and build files: backend/requirements.txt and requirements-dev.txt (pinned), frontend/package.json and package-lock.json, the two Dockerfiles, docker-compose.yml, docker-compose.dev.yml and .env.example.

6.2 Implementation of Core Capabilities

Capability	Requirement (s)	Main implementation components
Sign-in, sessions, passwords	FR-01, FR-02	AuthService, ForcePasswordChangeMiddleware, login and password views, django_session, users
User administration and CSV import	FR-03, FR-04	UserAdminService, UserImportService, user and import views, SCR-11, SCR-12, `users`
Report a fault	FR-05	SubmitRequestPage (SCR-04), RequestListCreateView, TransitionService.submit_request, maintenance_requests
Track requests and answer clarifications	FR-06, FR-07	PermissionPolicy.scope_requests, request list and detail views, timeline from audit_logs, SCR-03, SCR-05
Triage	FR-08, FR-09	TriageQueueView, RequestTransitionView, TransitionService.execute (REQUEST_CLARIFICATION, APPROVE, REJECT), SCR-06
Assign and reassign	FR-10, FR-11	DispatchPage (SCR-07), TransitionService.execute (ASSIGN, REASSIGN), work_orders with partial unique index
Technician work	FR-13 to FR-15	TechnicianWorkQueueView, RequestTransitionView, TransitionService.execute (START_WORK, COMPLETE_WORK), SCR-09
Verify, close, reopen	FR-12	VerificationPage (SCR-08), TransitionService.execute (VERIFY_CLOSE, REOPEN)
Dashboard	FR-16	ReportingService, DashboardView, SCR-10
Audit recording and review	FR-17, FR-18	AuditService, trigger and privilege migration, audit views, SCR-05, SCR-13

Validation happens in three layers: the client forms (for convenience), the DRF serialisers (shape and types) and the service rules (business rules), with database constraints underneath as a last check. Business rules are enforced by `TransitionRule` and `TransitionService`, not by views. `Every endpoint checks access through the permission classes and PermissionPolicy.` Data is saved through the Django ORM inside `transaction.atomic`. Errors follow one set of codes:

Status	Meaning	Client behaviour
401	Not authenticated or session ended	Client redirects to sign-in
403	Role not permitted for the function	Access-denied page
404	Object absent or outside the caller's scope	Not-found page
409	State conflict: expected status differs, or another user acted first	Message showing the current status; screen refreshes
422	Validation failure	Field-level messages

Status	Meaning	Client behaviour
429	Throttled	Retry-later message

6.3 APIs, Integrations and External Dependencies

The system uses no external services, and nothing is mocked or simulated. Its only interface is the REST API below, which my own web client uses. The OpenAPI 3 schema is generated by drf-spectacular into docs/openapi.yaml, and schema.d.ts is generated from it. Lists return total_count, limit, offset and results, with at most 50 rows on a page. Libraries such as Django, DRF, argon2-cffi and React are dependencies, not integrations.

Method and path	Roles	Req.	Purpose
POST /api/auth/login; POST /api/auth/logout; GET /api/auth/csrf	All	FR-01	Sign in and out; issue the CSRF token
POST /api/auth/password	All	FR-02	Change password (forced or voluntary)
GET /api/me	All	FR-01	Current user and role
GET, POST /api/users/; GET, PATCH /api/users/{id}/	Admin	FR-03	List, create and edit users, including deactivate and reactivate
POST /api/users/{id}/reset-password/	Admin	FR-03	Issue a new temporary password
POST /api/users/import-csv/	Admin	FR-04	All-or-nothing CSV import
GET /api/locations/; GET /api/categories/	All, scoped	FR-05	Form options
GET /api/residences/	Supervisor, Admin	FR-03, FR-08	Residences for filters and user forms
POST /api/requests/	Resident, Warden	FR-05	Submit (idempotent by client token)
GET /api/requests/; GET /api/requests/{id}/	Role-scoped	FR-06, FR-08	List and detail
GET /api/requests/{id}/timeline/	Role-scoped	FR-06, FR-18	Timeline and history
POST /api/requests/{id}/transition/	Per Table 5.1	FR-07, FR-09 to FR-12, FR-14, FR-15	Every state change: action, expected_status, payload
GET /api/triage-queue/	Supervisor	FR-08	Submitted requests, oldest first
GET /api/technicians/	Supervisor	FR-10	Active Technicians with open-work counts
GET /api/work-queue/	Technician	FR-13	Own active assignments
GET /api/dashboard/	Warden, Supervisor	FR-16	Role-scoped aggregates
GET /api/audit-logs/	Admin	FR-18	Filtered audit search
GET /api/health/; GET /api/schema/	Public	NFR-09	Health check; OpenAPI schema

6.4 AI / Machine-Learning Component

Not applicable. The product has no AI or machine-learning component. I left AI/ML out in the proposal and in Section 3.3.

AI tools used to build this project. I used AI coding assistants, Google Antigravity and Claude Code, for code, tests, diagrams and report text. docs/AGENT_BRIEF.md records how I worked with them. I checked their output by running the automated tests in Section 7, trying the system in the browser, and reading the changes before accepting them. [Draft for you to confirm or correct before submission: list every tool you used, including any used for the Phase documents, and follow the university's rules on declaring AI use.]

7. VERIFICATION AND TESTING

7.1 Test Approach

Most checks are automated and run against a real PostgreSQL 16 database, so the locks, constraints and audit trigger are actually exercised. On the final build, 124 pytest tests cover the API, every endpoint for every role, all 450 combinations of request state, action and role, parallel clients, direct SQL attacks on the constraints, and randomised operation sequences. 10 Vitest tests cover the client, 6 Playwright journeys run in the browser against the Docker stack (in Chromium, Chrome and Edge), and a Locust profile measures load. The screenshots in Section 5.5 were captured by script from the running system. Local test runs used Python 3.14; the containers and CI use Python 3.12.

Every success criterion points to named evidence (Section 9). Test code is in backend/tests/, frontend/src/, frontend/e2e/ and loadtests/. Summaries of the runs are in docs/evidence/. The GitHub Actions workflow runs lint, type checks, tests, dependency audits and the clean-stack journeys on every push.

7.2 Functional and Business-Rule Verification

Actual Result and Status come from the final test run. Each row names the test file that holds the test.

Test ID	Requirement / rule	Test scenario	Type	Expected result	Actual result	Status
TC-01	FR-01; AC-01.1	Sign in with valid credentials	API	Session created; role home screen returned	As expected. test_auth.py	Pass
TC-02	FR-01; AC-01.2	Wrong password; unknown email	API	Identical 401 message; failure event recorded	As expected. test_auth.py	Pass

Test ID	Requirement / rule	Test scenario	Type	Expected result	Actual result	Status
TC-03	FR-01; AC-01.3; NFR-02	Six failed attempts within one minute	API	Sixth attempt returns 429	As expected. test_auth.py	Pass
TC-04	FR-01; AC-01.4; BR-07	Deactivated user signs in; user deactivated while signed in	API	Sign-in refused; open session rejected on next call	As expected. test_auth.py	Pass
TC-05	FR-02	Temporary password sign-in; weak then strong new password	API	Forced change; weak rejected; strong accepted	As expected. test_auth.py	Pass
TC-06	FR-03; BR-07	Deactivate a Technician with an active work order; demote the last Admin	API	409 in both cases; nothing changes	As expected. test_users.py	Pass
TC-07	FR-04; AC-04.1	Import a valid CSV of N rows	API	N accounts created; one-time credentials file	As expected. test_users.py	Pass
TC-08	FR-04; AC-04.2	CSV with an invalid row and a duplicate email	API	No accounts created; every bad row listed	As expected. test_users.py	Pass
TC-09	FR-04; AC-04.3	CSV over 2 MB or 2,000 rows or non-UTF-8; non-Admin caller	API	Rejected before processing; 403	As expected. test_users.py	Pass
TC-10	FR-05; AC-05.1	Valid submission	API / E2E	Stored as Submitted; tracking ID; audit event	As expected. test_workflow.py	Pass
TC-11	FR-05; AC-05.2; BR-09	Description of 9 and 2,001 characters; missing category; other-residence location	API	422 with field errors; nothing stored	As expected. test_workflow.py	Pass
TC-12	FR-05; AC-05.3	Same client submission token sent twice	API	One record; original returned	As expected. test_workflow.py	Pass
TC-13	FR-05; AC-05.4	Supervisor, Technician or Admin submits	API	403	As expected. test_workflow.py	Pass
TC-14	FR-06; BR-05	Resident lists own requests; reads another Resident's request	API	Own only; other returns 404	As expected. test_permissions.py	Pass
TC-15	FR-07; BR-02	Answer a clarification; answer in the wrong state; non-owner answers	API	Back to Submitted; 409; 404	As expected. test_workflow.py	Pass
TC-16	FR-08	Queue order, filters and page size; non-Supervisor	API	Oldest first; at most 50 per page; 403	As expected. test_regressions.py	Pass
TC-17	FR-09; AC-09.1; BR-08	Approve with and without priority	API	Approved; 422 without priority	As expected. test_workflow.py	Pass
TC-18	FR-09; AC-09.2; BR-02	Reject with a whitespace-only reason; with a valid reason	API	422; Rejected, reason visible to Resident	As expected. test_workflow.py	Pass

Test ID	Requirement / rule	Test scenario	Type	Expected result	Actual result	Status
TC-19	FR-07; FR-09; AC-09.3	Clarification loop: ask, answer, approve	E2E	States follow Table 5.1	As expected. frontend/e2e/journeys.spec.ts	Pass
TC-20	FR-09; AC-09.4	Two Supervisors approve and reject the same request simultaneously; stale expected status	Concurrency	One succeeds, the other 409; nothing corrupted	As expected. test_concurrency.py	Pass
TC-21	FR-10; AC-10.1	Assign an Approved request to an active Technician	API	Assigned; work order created; event written	As expected. test_workflow.py	Pass
TC-22	FR-10; AC-10.2	Assign to an inactive user and to a non-Technician	API	422; state unchanged	As expected. test_workflow.py	Pass
TC-23	FR-10; AC-10.4; BR-01; NFR-04; SC-3	20 simultaneous assignments of one request, repeated 50 times	Concurrency	Exactly one success and 19 conflicts per run; one active work order	50 runs of 20 clients: 1 success and 19 conflicts each time. test_concurrency.py	Pass
TC-24	FR-10; AC-10.3; NFR-03	Fault injected after the work-order insert and before commit	Integration	Complete rollback; no event	As expected. test_database.py	Pass
TC-25	FR-10; BR-01; NFR-08	Insert a second active work order with direct SQL	Database	Rejected by the partial unique index	As expected. test_database.py	Pass
TC-26	FR-11; AC-11.1; BR-03	Reassign an In Progress request	API	Old work order ends REASSIGNED; new one created; Assigned; event names both Technicians	As expected. test_workflow.py	Pass
TC-27	FR-11; AC-11.2; AC-11.3	Same Technician; wrong state; former Technician acts afterwards	API	422; 409; 404	As expected. test_workflow.py	Pass
TC-28	FR-13; BR-04	Technician queue contents and fields; another Technician's request	API	Own only, ordered; no resident fields; 404	As expected. test_permissions.py	Pass
TC-29	FR-14	Start from Assigned and from Reopened; start from other states	API	In Progress; 409	As expected. test_workflow.py	Pass
TC-30	FR-15; AC-15.1; BR-02	Completion notes of 19 and 20 characters	API	19 gives 422; 20 gives Work Completed	As expected. test_workflow.py	Pass
TC-31	FR-15; AC-15.2; AC-15.3	Empty and whitespace-only notes; double submission	API / E2E	422; one transition, second call 409	As expected. test_workflow.py	Pass
TC-32	FR-12; AC-12.1	Verify a Work Completed request	API	Verified & Closed; closed_at set; work order ends COMPLETED	As expected. test_workflow.py	Pass

Test ID	Requirement / rule	Test scenario	Type	Expected result	Actual result	Status
TC-33	FR-12; AC-12.2; BR-02	Reopen with and without feedback	API	Reopened and feedback visible; 422 without	As expected. test_workflow.py	Pass
TC-34	FR-12; AC-12.3; BR-06; BR-10	Act on a terminal request; Technician attempts to verify	API	409; 403	As expected. test_workflow.py	Pass
TC-35	SC-1; FR-05; FR-07; FR-09; FR-10; FR-11; FR-12; FR-14; FR-15	Every state × action × role combination generated from Table 5.1	Unit / API	All legal transitions succeed; all others refused with no change	All 450 combinations behave as Table 5.1 says. test_matrices.py	Pass
TC-36	FR-16; AC-16.1; BR-04	Warden dashboard scope and write attempts	API	Own residence only; no resident identity; writes 403	As expected. test_permissions.py	Pass
TC-37	FR-16; AC-16.2; AC-16.3	Aggregates compared with direct SQL on seeded data; Supervisor scope	Integration	Equal; all residences visible	As expected. test_reporting_audit.py	Pass
TC-38	FR-17; AC-17.1	Count audit events for successful and refused transitions	Integration	Exactly one per success; none for refusal	As expected. test_reporting_audit.py	Pass
TC-39	FR-17; AC-17.2; NFR-05; SC-4	UPDATE, DELETE and TRUNCATE on audit_logs as the application role	Database	All three fail	All three fail as nrpj_app, also confirmed on the running stack. test_database.py	Pass
TC-40	FR-17; AC-17.3	Failed sign-in, role change, deactivation, password reset, bulk import	Integration	Each produces an event	As expected. test_reporting_audit.py	Pass
TC-41	FR-18	Supervisor reads request history; Admin searches; other roles call the global log	API	Allowed for Supervisor and Admin; 403 otherwise	As expected. test_reporting_audit.py	Pass
TC-42	SC-1	Full journey: Resident reports, Supervisor approves and assigns, Technician completes, Supervisor verifies, Resident sees Closed	E2E	Completes in the browser with correct status at each step	As expected. frontend/e2e/journeys.spec.ts	Pass

7.3 Non-Functional and Security Verification

Test ID	Requirement	Scenario	Method	Expected result	Actual result	Status
NT-01	NFR-01; SC-2	Authorisation matrix: every endpoint for every role, generated from a permission table, plus out-of-scope objects	API	No unauthorised call succeeds; unauthenticated 401, forbidden role 403, out-of-scope object 404	Every endpoint for every role, and out-of-scope objects: no unauthorised success. test_matrices.py	Pass
NT-02	NFR-02	Inspect stored passwords; inspect cookies and CSRF behaviour	Database / API	Argon2id hashes only; HttpOnly and SameSite=Lax; write without CSRF token refused	As expected. test_auth.py	Pass
NT-03	NFR-03	Randomised operation sequences followed by the invariant checker (INV-1 to INV-6)	Integration	All invariants hold after every run	All invariants held after each of 150 random actions, for 5 seeds. test_invariants.py	Pass
NT-04	NFR-06; SC-5	Load test: 30 simulated users, 5,000 seeded requests, Docker Compose on the reference machine (Section 8)	Load (Locust)	95% of API requests within 1 s; dashboard first load within 3 s	2,939 requests, 0 failures. 95% within 130 ms, slowest 720 ms; dashboard 95% within 140 ms. docs/evidence/nt-04-locust-summary.txt	Pass
NT-05	NFR-07	Viewport at 360 px; latest Chrome and Edge; Lighthouse accessibility on SCR-01, 04, 06, 09, 10	Manual / Lighthouse	No horizontal scroll; core flows work; score at least 90	No sideways scroll at 360 px. All 6 journeys pass in Chrome 156 and Edge 156. Lighthouse 100 on all five screens. docs/evidence/nt-05-lighthouse.txt	Pass
NT-06	NFR-08	Attempt each constraint violation directly in SQL (CHECK, FK, unique)	Database	Every violation rejected by the database	As expected. test_database.py	Pass
NT-07	NFR-09; SC-5	Clean clone, three documented commands, full test suite	CI	System starts; tests pass	The three steps built and started the system from a clean clone of the v1.0.0 tag, and all journeys passed against it. It also started with changed passwords. The CI clean-stack job repeats this on each push.	Pass
NT-08	NFR-10	Lint, TypeScript type checks and a repository check that status is assigned only inside TransitionService	CI	All pass	Ruff, ESLint and TypeScript pass. Status is assigned only in TransitionService. test_repository.py	Pass
NT-09	NFR-11	Response header check; pip-audit and npm audit (production dependencies)	CI	Required headers present; no unaddressed High or Critical finding	All four headers present. pip-audit and npm audit report no known vulnerability in production packages. docs/evidence/	Pass

Test ID	Requirement	Scenario	Method	Expected result	Actual result	Status
ST-01	NFR-01; NFR-11	SQL-injection strings in text fields and filter parameters	API	Stored or ignored literally; no error, no data leakage	Hostile values ignored or stored literally, no error. test_reporting_audit.py, test_regressions.py	Pass
ST-02	NFR-11	Description containing script markup, viewed by Supervisor and Resident	E2E	Rendered as inert text	Markup shown as text and no script ran. test_regressions.py, frontend/e2e/journeys.spec.ts	Pass
ST-03	NFR-01; BR-08	Mass assignment: send status, role, resident_id or priority in a Resident's request	API	Fields ignored or rejected; no state or role change	As expected. test_permissions.py	Pass
ST-04	NFR-01; BR-05	Enumerate request IDs as Resident and as Technician	API	Only permitted records returned; others 404	As expected. test_permissions.py	Pass
ST-05	NFR-02; BR-07	Reuse a session cookie after sign-out and after deactivation	API	401	As expected. test_auth.py	Pass
ST-06	BR-04	Scan every Warden and Technician JSON response for resident name, email and student number	API	None present	As expected. test_permissions.py	Pass

7.4 Defects and Known Issues

These significant defects were found by a full code audit before submission, using scripted probes against a throwaway database and the running stack. All are fixed, and each fix has a regression test in backend/tests/test_regressions.py unless the table names another check. None remains open.

Ref	Defect	Found by	Resolution	Status
DEF-01	A temporary password did not block the API: the forced change existed only in the browser.	API probe	Server middleware blocks everything but the password change.	Closed
DEF-02	Failed sign-ins locked out everyone on one IP, for up to 24 hours, even with a correct password.	API probe	Returned to the rule in NFR-02: 5 failures a minute, per account and per IP.	Closed
DEF-03	Deactivating a Technician could cross with an assignment, leaving an inactive Technician with work (14 of 25 runs).	Race probe	Deactivation takes the same row lock as assignment.	Closed
DEF-04	Non-numeric filter values returned a server error.	ST-01 probe	Filters parse safely and ignore bad values.	Closed

Ref	Defect	Found by	Resolution	Status
DEF-05	Two simultaneous submits with one token could return a server error (9 of 48 calls).	Race probe	The constraint error now returns the original request.	Closed
DEF-06	A refused user change still wrote an audit event and ended the user's session.	API probe	User changes run in one transaction.	Closed
DEF-07	The triage queue was newest first, had no 50-row limit and no residence filter.	TC-16 probe	Added /api/triage-queue/ and pagination on every list.	Closed
DEF-08	A Warden's report that needed clarification could never be answered.	Code review	The owner answers, whatever their role.	Closed
DEF-09	A role that may never act got 409, not 403, which revealed the request's state.	API probe	Role is checked before state.	Closed
DEF-10	Technicians were sent to the 403 page from their own jobs, so they could not read reopen feedback.	Code review	Route opened; covered by TC-42.	Closed
DEF-11	A short CSV row returned a server error, and the role and residence columns were ignored.	API probe	Row-level errors; all FR-04 columns honoured.	Closed
DEF-12	The emergency notice and the “Action required” section were missing.	Browser check	Both added; covered by journeys.spec.ts.	Closed
DEF-13	An expired session left the user on a stale screen with no notice.	Browser check	Redirect to sign-in with a notice; covered by journeys.spec.ts.	Closed
DEF-14	Two-factor sign-in, outside the agreed scope, had no limit on code guesses.	API probe	Removed; Section 3.3 keeps it Excluded.	Closed
DEF-15	The proxy container reported unhealthy: its health check used localhost, which is IPv6 in that image, while nginx listens on IPv4.	Container status	Check uses 127.0.0.1; docker compose up --wait now passes.	Closed

8. SYSTEM SETUP AND OPERATION

Prerequisites: Git and Docker Engine with Compose v2. Reference machine for NFR-06: AMD Ryzen 5 5625U, 7.3 GB of memory, Windows 11 Pro, Docker Desktop with 3.5 GB of memory for containers.

Start the system (three commands):

Step	Command	Effect
1	`git clone https://github.com/KamogeloBantsheng/NRPJ740-PROJECT.git && cd NRPJ740-PROJECT`	Fetch the tagged release (Section 11.1).
2	`cp .env.example .env`	Create local configuration. Replace placeholder values before use.
3	`docker compose up --build`	Builds images, starts the database, runs migrations and seed data, then starts the API and proxy. Open http://localhost:8080. Use docker compose up --build -d to get the terminal back.

Configuration (`.env`; placeholder values only):

Variable	Example	Purpose
`DJANGO_SECRET_KEY`	`change-me`	Signing key; must be unique per deployment.
`DJANGO_DEBUG`	`false`	Disable debug output.
`DB_OWNER_PASSWORD`, `DB_APP_PASSWORD`	`change-me`	Passwords of the migration (owner) role and the runtime (application) role.
`SESSION_COOKIE_AGE`	`28800`	Session lifetime in seconds (8 hours).
`SEED_DEMO_DATA`	`true`	Seed one demo account per role and sample requests (evaluation profile only).
`DEMO_PASSWORD`	`change-me`	Password of the demo accounts.
`DB_STATEMENT_TIMEOUT_MS`	`30000`	Longest time one SQL statement may run.

Demo accounts One per role, and a second Technician for reassignment. The password is the value of DEMO_PASSWORD in .env.

Role	Email	Password
Resident	resident@example.test	value of DEMO_PASSWORD
Supervisor	supervisor@example.test	as above
Technician	technician@example.test	as above
Technician	technician2@example.test	as above
Warden	warden@example.test	as above
Admin	admin@example.test	as above

Operation: Stop with docker compose down; reset all data with docker compose down -v. Back up with docker compose exec -T db pg_dump -U postgres nrpj740_dev > backup.sql and restore with psql into an empty database (LIM-04); this round trip was tried on the final build.

Backend tests need PostgreSQL and a role that can create databases: install backend/requirements-dev.txt, set the DB_ variables as in .github/workflows/ci.yml, then run pytest backend/tests/. Run the client tests with npm test in frontend/, and the browser journeys with npx playwright test while the stack is running. For the load test, run docker compose exec api python manage.py seed_load_data on a throwaway database, then locust -f loadtests/locustfile.py --headless -u 30 -r 3 -t 3m --host http://localhost:8080.

9. SOLUTION EVALUATION

Success criterion	Outcome	Evidence
SC-1 Core workflow	Achieved	TC-35 (all 450 combinations), TC-42 and TC-19 pass in the browser.
SC-2 Access control	Achieved	NT-01, ST-03 to ST-06, TC-14, TC-28 and TC-36 pass. The one Warden write is the approved exception (CH-10).
SC-3 Concurrency and data integrity	Achieved	TC-23: 50 runs of 20 clients gave one success each. TC-20, TC-24, TC-25 and NT-03 pass, and the race in DEF-03 is closed.
SC-4 Accountability	Achieved	TC-38 to TC-40 pass. UPDATE, DELETE and TRUNCATE also failed for the application role on the running stack.
SC-5 Verified quality	Achieved	All automated tests pass; NT-04 measured 130 ms against a 1 second target; NT-05 and NT-07 pass.

10. REFERENCES

Django Software Foundation. (n.d.-a). *QuerySet API reference: select_for_update* (Django 5.2 documentation). https://docs.djangoproject.com/en/5.2/ref/models/querysets/#select-for-update

Django Software Foundation. (n.d.-b). *Download Django: Supported versions*. https://www.djangoproject.com/download/

OWASP Foundation. (n.d.). *Password storage cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

PostgreSQL Global Development Group. (n.d.-a). *Explicit locking* (PostgreSQL 16 documentation). https://www.postgresql.org/docs/16/explicit-locking.html

PostgreSQL Global Development Group. (n.d.-b). *Partial indexes* (PostgreSQL 16 documentation). https://www.postgresql.org/docs/16/indexes-partial.html

Republic of South Africa. (2013). *Protection of Personal Information Act 4 of 2013* (Government Gazette No. 37067). Government Printer.

11. APPENDICES

11.1 GitHub Repository

Repository URL: https://github.com/KamogeloBantsheng/NRPJ740-PROJECT

Submitted release tag and commit hash: v1.0.0 / 148e8a523adc8a4dca695dc1ebd459da3cb359b9

The repository is shared with the lecturer, Dr Silas Verkijika, as a collaborator.

11.2 Deployed System

Not applicable. The assessed system runs locally in containers (Section 5.8 and Section 8) and is not deployed to a server.

11.3 Additional Supporting Material

- Interview questions and answer summary for Problem evidence P-1 to P-4 (role, date, method): Phase 1 document, Section 9.
- Diagram source files (draw.io) and exported images, in `docs/diagrams/` in the repository.
- Generated OpenAPI schema and extended test and load-test output, in `docs/evidence/`.